

Aprendizaje por Refuerzo

Guillermo Carmona Martínez
`g.carmonamartinez@um.es`

Francisco José Cortes Delgado
`franciscojose.cortesd@um.es`

Víctor Saura Meseguer
`victor.sauram@um.es`

15 de marzo de 2026

Resumen

Este trabajo aborda dos problemas clásicos del aprendizaje por refuerzo.

La primera parte estudia el problema del **bandido de k -brazos**, un entorno no asociativo de toma de decisiones secuenciales bajo incertidumbre. Se implementan y comparan cinco algoritmos de tres familias distintas: ε -**Greedy** (con y sin inicialización optimista y con decaimiento), **UCB1** y **Softmax**. Todos emplean estimación incremental del promedio y se evalúan sobre distribuciones de recompensa Normal, Binomial y Bernoulli, analizando el equilibrio exploración-explotación mediante regret acumulado, recompensa promedio y porcentaje de selecciones óptimas. Los resultados muestran que **UCB1** obtiene el menor regret acumulado en todas las distribuciones estudiadas, confirmando la superioridad de la exploración guiada por incertidumbre frente a la exploración aleatoria de ε -Greedy.

La segunda parte extiende el estudio a **entornos complejos** con estado dinámico. Se implementan y comparan métodos de **control tabular** (Monte Carlo on/off-policy, SARSA y Q-Learning) en los entornos discretos Taxi-v3 y CliffWalking-v1, y métodos de **control con aproximación de función** (SARSA semi-gradiente y Deep Q-Learning con Experience Replay) en el entorno continuo LunarLander-v3. Respecto a los resultados, en los entornos discretos, los métodos TD superan claramente a Monte Carlo en velocidad de convergencia, y el contraste on/off-policy queda ilustrado en CliffWalking: Q-Learning aprende la ruta óptima mientras SARSA adopta una política más conservadora. En LunarLander-v3, la aproximación de función resulta imprescindible, y SARSA semi-gradiente alcanza mayor recompensa acumulada frente a DQN, aunque ninguno supera el umbral de resolución del entorno (200 puntos) en el horizonte de entrenamiento establecido.

Asimismo, en las conclusiones del Capítulo 2 se incluye una declaración explícita del uso de herramientas de inteligencia artificial durante el desarrollo del trabajo, detallando los ámbitos concretos en que fueron empleadas. Todo el código y los experimentos están disponibles en [GitHub](#) con reproducibilidad garantizada mediante semilla fija.

Índice general

1. Problema del Bandido de k-brazos	1
1.1. Introducción	1
1.2. Desarrollo	1
1.2.1. Enfoques y Métodos Utilizados	1
1.3. Algoritmos	2
1.4. Evaluación/Experimentos	3
1.4.1. Configuración experimental	4
1.4.2. Métricas	4
1.4.3. Resultados y Análisis Crítico	4
1.5. Conclusiones	9
2. Aprendizaje en Entornos Complejos	10
2.1. Introducción	10
2.2. Desarrollo	10
2.2.1. Definición Formal del Problema	10
2.2.2. Enfoques y Métodos Utilizados	11
2.3. Algoritmos	11
2.4. Evaluación/Experimentos	14
2.4.1. Escenarios experimentales	14
2.4.2. Configuración Hiperparámetros	14
2.4.3. Métricas	15
2.4.4. Resultados y Análisis Crítico	15
2.5. Conclusiones	19
2.5.1. Uso de Herramientas de Inteligencia Artificial	19

Capítulo 1

Problema del Bandido de k -brazos

1.1. Introducción

El problema del bandido de K -brazos [5] describe un escenario en el cual un agente debe elegir repetidamente entre k opciones diferentes. Al seleccionar una acción, el agente recibe una recompensa numérica proveniente de una distribución de probabilidad desconocida asociada a ese brazo. El objetivo fundamental es maximizar la recompensa total acumulada a lo largo de un número determinado de pasos de tiempo, sin conocimiento previo de qué brazo es el mejor.

La importancia y relevancia de este problema reside en su capacidad para modelar el dilema fundamental entre **exploración** y **explotación** [7]. Un equilibrio inadecuado entre ambas estrategias resulta costoso, podríamos ignorar la mejor opción de forma permanente si solo explotamos, o generar un alto arrepentimiento acumulado al explorar en exceso sin converger hacia el brazo óptimo. Este dilema constituye el núcleo del aprendizaje por refuerzo en su forma más simple. Además, el modelo tiene aplicaciones directas en problemas del mundo real, como ensayos clínicos médicos, marketing digital (pruebas A/B), sistemas de recomendación y optimización de redes de comunicaciones.

El objetivo de este informe es analizar y comparar cinco estrategias algorítmicas para resolver el problema del bandido multibrazo: **ϵ -Greedy**, **ϵ -Greedy con inicialización**, **ϵ -Decaimiento**, **UCB1** y **Softmax**. Se estudia su comportamiento sobre tres tipos de distribuciones de recompensa (Normal, Bernoulli y Binomial) y se evalúa cómo cada estrategia gestiona el equilibrio exploración-explotación para minimizar el arrepentimiento acumulado.

El informe se estructura en cinco secciones. La **Introducción** (presente sección) contextualiza el problema y define los objetivos. El **Desarrollo** define formalmente el modelo y describe las familias de métodos existentes en la literatura. La sección de **Algoritmos** detalla el pseudocódigo de cada uno de los cinco métodos implementados. La **Evaluación** presenta la configuración experimental, las métricas empleadas y el análisis crítico de los resultados. Por último, las **Conclusiones** resumen los hallazgos principales e identifican limitaciones y líneas de trabajo futuro.

1.2. Desarrollo

El bandido de K -brazos [5] es un proceso de decisión secuencial en el que, en cada instante $t = 1, 2, \dots, T$, un agente elige una acción $a_t \in \mathcal{A} = \{a_1, \dots, a_k\}$ y recibe una recompensa

$r_t \sim p(r \mid a_t)$ proveniente de una distribución desconocida.

Definición 1.2.1 (Valor de una acción). El valor real de una acción a es la recompensa esperada al seleccionarla: $q(a) = \mathbb{E}[R_t \mid A_t = a]$.

Como $q(a)$ es desconocido, el agente debe estimarlo a partir de la experiencia mediante la estimación incremental del promedio:

$$Q_{n+1}(a) = Q_n(a) + \frac{1}{N_n(a)}(r_n - Q_n(a)) \quad (1.1)$$

donde $N_n(a)$ es el número de veces que el brazo a ha sido seleccionado hasta el paso n . Esta fórmula opera en tiempo y espacio $O(1)$ por paso, sin necesidad de almacenar el historial completo de recompensas. El objetivo es maximizar la recompensa acumulada $\sum_{t=1}^T r_t$, lo que equivale a minimizar el *arrepentimiento esperado*:

$$\mathbb{E}[R_T] = T q^* - \sum_{t=1}^T \mathbb{E}[r_t], \quad q^* = \max_{a \in \mathcal{A}} q(a) \quad (1.2)$$

El reto central es el equilibrio entre **exploración** (probar brazos poco conocidos para mejorar las estimaciones) y **explotación** (seleccionar el brazo con mayor recompensa esperada conocida).

1.2.1. Enfoques y Métodos Utilizados

A diferencia del aprendizaje supervisado, que proporciona **retroalimentación instructiva** (la acción correcta es conocida), el bandido opera con **retroalimentación evaluativa**, es decir, el agente solo conoce la calidad de la acción tomada, no si existía una mejor [7]. Es además un entorno **no asociativo**, sin estado observable que condicione la decisión, lo que lo convierte en el modelo más simple del aprendizaje por refuerzo. Se asume que las distribuciones son **estacionarias** ($p(r \mid a)$ constante en el tiempo).

Desde su formulación, han surgido cuatro grandes familias de soluciones, de las cuales haremos uso para abordar el problema:

1. **Métodos acción-valor con exploración heurística.** Estiman $q(a)$ mediante la Ecuación (1.1) y aplican una política que intercala explotación y exploración. El **ϵ -greedy** explora al azar con probabilidad fija ϵ [7], mientras que el **ϵ -decaimiento** reduce ϵ gradualmente para favorecer la explotación conforme avanza el aprendizaje.

2. **Métodos de índice (*Upper Confidence Bound*, UCB).** Seleccionan la acción que maximiza un índice que combina valor estimado e incertidumbre. UCB1 [1] aplica el principio de *optimismo ante la incertidumbre*.
3. **Métodos de ascenso del gradiente** En lugar de una decisión binaria explorar/explotar, asignan a cada brazo una probabilidad de selección proporcional a su valor estimado mediante la distribución de Gibbs [7]. La temperatura τ regula la concentración, de manera que $\tau \rightarrow 0$ equivale a la política greedy y $\tau \rightarrow \infty$ produce selección uniforme.
4. **Métodos bayesianos.** Mantienen una distribución posterior sobre los parámetros de cada brazo, actualizada con la regla de Bayes. El **Muestreo de Thompson** [8, 6] muestrea de la posterior y selecciona el brazo con mayor valor estimado, logrando una exploración adaptativa eficaz.

Justificación del trabajo

La selección de los cinco algoritmos responde a un criterio de comparabilidad, todos comparten el mismo núcleo de estimación (la actualización incremental de la Ecuación (1.1)) y se diferencian exclusivamente en su **política de selección**. Esto permite aislar el efecto de cada estrategia de exploración-explotación sobre el arrepentimiento, manteniendo constante el resto del sistema.

Los cinco algoritmos seleccionados cubren el espectro de estrategias más estudiado en la literatura, desde la exploración aleatoria más simple (ε -Greedy) hasta la dirigida por incertidumbre (UCB1), pasando por variantes que adaptan la exploración con el tiempo (ε -Decaimiento, inicialización forzada) o la distribuyen proporcionalmente (Softmax). Para evaluar su robustez, el análisis se extiende a tres distribuciones de recompensa con propiedades estadísticas distintas, la Normal, habitual en la literatura experimental [7], la Bernoulli, de interés en aplicaciones reales como A/B testing o ensayos clínicos y la Binomial, que generaliza la anterior e introduce un rango de recompensa intermedio. Esta elección permite comprobar si las conclusiones sobre el rendimiento relativo de los algoritmos se mantienen al cambiar la naturaleza de las recompensas.

1.3. Algoritmos

Los cinco algoritmos comparten el núcleo de estimación de la Ecuación (1.1) y se diferencian únicamente en su política de selección. El Algoritmo 1 resume la actualización incremental común a todos ellos.

Algorithm 1 Actualización incremental del valor estimado

Require: Brazo seleccionado a , recompensa observada r

- 1: $N(a) \leftarrow N(a) + 1$
 - 2: $Q(a) \leftarrow Q(a) + \frac{1}{N(a)}(r - Q(a))$
-

ε -Greedy

El ε -Greedy [7] es el método más simple para incorporar exploración. En cada paso, con probabilidad ε selecciona un

brazo uniformemente al azar (exploración) y con probabilidad $1 - \varepsilon$ explota el brazo de mayor valor estimado, los empates se rompen aleatoriamente para evitar sesgo sistemático.

Su principal limitación es que para $\varepsilon = 0$ el agente explota desde el primer paso sin garantía de haber visitado todos los brazos, pudiendo converger a un subóptimo local de forma permanente. Para $\varepsilon > 0$, la exploración persiste indefinidamente incluso cuando el agente ya ha identificado el brazo óptimo, generando arrepentimiento residual proporcional a ε en cada paso.

Se incluye como **línea base**, su comportamiento es bien conocido y cualquier mejora medible sobre él cuantifica directamente el beneficio de estrategias más sofisticadas.

Algorithm 2 ε -Greedy

Require: $k, \varepsilon \in [0, 1]$

- 1: $N(a) \leftarrow 0, Q(a) \leftarrow 0$ **para todo** a
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: **if** $\text{Uniforme}(0, 1) < \varepsilon$ **then**
 - 4: $A_t \leftarrow$ brazo aleatorio de $\{1, \dots, k\}$ $\triangleright$ Exploración
 - 5: **else**
 - 6: $A_t \leftarrow \arg \max_a Q(a)$, empates rotos aleatoriamente $\triangleright$ Explotación
 - 7: **end if**
 - 8: Recibir R_t ; actualizar $N(A_t)$ y $Q(A_t)$ (Alg. 1)
 - 9: **end for**
-

ε -Greedy con inicialización forzada

Esta variante introduce una **fase de arranque**, antes de aplicar la política ε -greedy, visita cada brazo exactamente una vez en orden aleatorio. Cuando comienza la fase de explotación, todos los brazos tienen al menos una estimación $Q(a)$, eliminando el riesgo de convergencia a un subóptimo local por falta de exploración inicial.

A diferencia de los *valores iniciales optimistas* [7] que fijan $Q_1(a)$ artificialmente alto para inducir exploración temprana mediante la propia política greedy, aquí la exploración inicial es **explícita y exhaustiva**, se garantiza visitar los k brazos independientemente del valor de ε . Esto es especialmente ventajoso para $\varepsilon = 0$, donde el greedy puro puede explotar correctamente tras la fase de arranque sin incurrir en exploración residual permanente.

ε -Decaimiento

El ε -Decaimiento mantiene la política ε -greedy pero reduce ε con el tiempo. La implementación usa un **decaimiento inversamente proporcional**:

$$\varepsilon_t = \max\left(\varepsilon_{\min}, \frac{\varepsilon_0}{1 + \lambda t}\right) \quad (1.3)$$

donde ε_0 es el valor inicial, $\lambda > 0$ la tasa de decaimiento y $\varepsilon_{\min} \geq 0$ un umbral que evita que la exploración desaparezca por completo.

Se elige el decaimiento inversamente proporcional frente a las dos alternativas principales por su comportamiento más adecuado, el *lineal* ($\varepsilon_t = \varepsilon_0 - \lambda t$) llega a cero en un tiempo fijo,

Algorithm 3 ε -Greedy con inicialización forzada

Require: $k, \varepsilon \in [0, 1]$

- 1: $N(a) \leftarrow 0, Q(a) \leftarrow 0$ **para todo** a
- 2: **for** $t = 1, 2, \dots, T$ **do**
- 3: **if** $\exists a : N(a) = 0$ **then**
- 4: $A_t \leftarrow$ brazo aleatorio de $\{a : N(a) = 0\}$ $\triangleright$ Fase de arranque
- 5: **else if** $\text{Uniforme}(0, 1) < \varepsilon$ **then**
- 6: $A_t \leftarrow$ brazo aleatorio de $\{1, \dots, k\}$ $\triangleright$ Exploración
- 7: **else**
- 8: $A_t \leftarrow \arg \max_a Q(a)$, empates rotos aleatoriamente $\triangleright$ Explotación
- 9: **end if**
- 10: Recibir R_t ; actualizar $N(A_t)$ y $Q(A_t)$ (Alg. 1)
- 11: **end for**

pudiendo cortar la exploración antes de identificar el óptimo, el *exponencial* ($\varepsilon_t = \varepsilon_0 e^{-\lambda t}$) decae muy rápidamente al inicio, dejando poco margen para explorar. El inversamente proporcional descende de orden $O(1/t)$, garantizando exploración suficiente en las primeras etapas y convergencia progresiva hacia la explotación, lo que se alinea con la estrategia óptima en entornos estacionarios de horizonte largo [7]. Al igual que la variante anterior, incluye fase de arranque antes de aplicar el decaimiento.

Algorithm 4 ε -Decaimiento (inversamente proporcional)

Require: $k, \varepsilon_0 \in (0, 1], \lambda > 0, \varepsilon_{\min} \geq 0$

- 1: $N(a) \leftarrow 0, Q(a) \leftarrow 0$ **para todo** a ; $t \leftarrow 0$
- 2: **for** $\text{paso} = 1, 2, \dots, T$ **do**
- 3: **if** $\exists a : N(a) = 0$ **then**
- 4: $A \leftarrow$ brazo aleatorio de $\{a : N(a) = 0\}$ $\triangleright$ Fase de arranque
- 5: **else**
- 6: $\varepsilon_t \leftarrow \max\left(\varepsilon_{\min}, \frac{\varepsilon_0}{1 + \lambda t}\right)$; $t \leftarrow t + 1$
- 7: Seleccionar A según Alg. 2 con $\varepsilon = \varepsilon_t$
- 8: **end if**
- 9: Recibir R ; actualizar $N(A)$, $Q(A)$ (Alg. 1)
- 10: **end for**

UCB1

UCB1 [1] implementa el principio de *optimismo ante la incertidumbre*, en lugar de explorar aleatoriamente, selecciona en cada paso el brazo con mayor **límite superior de confianza**:

$$\text{UCB1}(a) = \underbrace{Q(a)}_{\text{explotación}} + c \underbrace{\sqrt{\frac{\ln t}{N(a)}}}_{\text{bonus de exploración}} \quad (1.4)$$

El **término de explotación** $Q(a)$ es la estimación actual del valor del brazo. El **bonus de exploración** cuantifica la incertidumbre, decrece cuando $N(a)$ crece (más visitas $\Rightarrow$ menor incertidumbre) y crece cuando t aumenta (brazos poco visitados se vuelven relativamente más inciertos). El parámetro $c \geq 0$ escala la amplitud del intervalo, es decir, con c pequeño

el algoritmo tenderá a explotar porque dominará $Q(a)$, mientras que con c grande el bonus domina y fuerza exploración incluso sobre brazos ya bien estimados.

Destacar que la elección del bonus $\sqrt{\ln t / N(a)}$ no es arbitraria, aplicando la **desigualdad de Hoeffding** [1] sobre recompensas acotadas, se puede demostrar que la probabilidad de que el valor real $q(a)$ supere el límite $\text{UCB1}(a)$ es como mucho t^{-2} , es decir, disminuye conforme avanza el tiempo. Esto justifica que el índice sea un límite superior de confianza fiable. Como resultado, Auer et al. [1] prueban que el arrepentimiento crece como $O(\ln T)$, lo que coincide en tasa de crecimiento con la cota inferior teórica de Lai y Robbins [2]. Por lo tanto, a diferencia de ε -Greedy, UCB1 es una **política determinista**, no explora al azar, sino que siempre elige el brazo con mayor índice UCB1, por lo que la exploración está guiada por la incertidumbre acumulada de cada brazo.

Algorithm 5 UCB1

Require: $k, c \geq 0$

- 1: $N(a) \leftarrow 0, Q(a) \leftarrow 0$ **para todo** a
- 2: **for** $t = 1, 2, \dots, T$ **do**
- 3: **if** $\exists a : N(a) = 0$ **then**
- 4: $A_t \leftarrow$ brazo aleatorio de $\{a : N(a) = 0\}$ $\triangleright$ Fase inicial
- 5: **else**
- 6: $t_{\text{total}} \leftarrow \sum_a N(a)$
- 7: $A_t \leftarrow \arg \max_a \left[Q(a) + c \sqrt{\frac{\ln t_{\text{total}}}{N(a)}} \right]$
- 8: **end if**
- 9: Recibir R_t ; actualizar $N(A_t)$, $Q(A_t)$ (Alg. 1)
- 10: **end for**

Softmax (Gibbs)

El Softmax [7] se incluye como representante de los **métodos de ascenso del gradiente**, a diferencia de los métodos anteriores, que separan exploración y explotación de forma binaria, asigna a cada brazo una probabilidad de selección *proporcional* a su valor estimado mediante la **distribución de Gibbs**:

$$\pi_t(a) = \frac{\exp(Q(a)/\tau)}{\sum_{b=1}^k \exp(Q(b)/\tau)} \quad (1.5)$$

donde $\tau > 0$ es la **temperatura**. A diferencia de los métodos ε -Greedy, la exploración es **proporcional a los valores estimados**, brazos con mayor $Q(a)$ reciben mayor probabilidad, pero ninguno queda completamente excluido. El parámetro τ regula la concentración: $\tau \rightarrow 0$ concentra toda la probabilidad en el máximo (política greedy) y $\tau \rightarrow \infty$ produce selección uniforme. A diferencia del ε -Decaimiento, la exploración **no decrece automáticamente** con el tiempo a menos que τ se reduzca de forma explícita, lo que puede suponer una desventaja en entornos estacionarios de horizonte largo.

Para evitar **desbordamiento numérico** cuando $Q(a)/\tau$ toma valores grandes, la implementación sustrae el máximo antes del cálculo: $\exp((Q(a) - Q_{\max})/\tau)$. Esta operación no altera las probabilidades resultantes pero garantiza estabilidad, y que no sucedan fallos de cálculos o memoria.

1.4. Evaluación/Experimentos

Algorithm 6 Softmax (Gibbs)**Require:** $k, \tau > 0$

```

1:  $N(a) \leftarrow 0, Q(a) \leftarrow 0$  para todo  $a$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $Q_{\max} \leftarrow \max_a Q(a)$ 
4:    $\pi(a) \leftarrow \frac{\exp\left(\frac{Q(a) - Q_{\max}}{\tau}\right)}{\sum_{b=1}^k \exp\left(\frac{Q(b) - Q_{\max}}{\tau}\right)}$  para todo  $a$ 
5:    $A_t \leftarrow$  brazo muestreado con probabilidades  $\pi$ 
6:   Recibir  $R_t$ ; actualizar  $N(A_t), Q(A_t)$  (Alg. 1)
7: end for

```

1.4.1. Configuración experimental

Los experimentos se han implementado en Python y Jupyter Notebooks como entorno de ejecución. Se han evaluado los cinco algoritmos en **tres entornos de distribución** distintos con $k = 10$ brazos, $T = 1000$ pasos (excepto UCB1, con $T = 500$) y 500 ejecuciones independientes, con semilla fija (`seed = 42`) para reproducibilidad. El análisis de hiperparámetros de cada algoritmo usa $T = 1000$ pasos, excepto para UCB1 donde se emplea $T = 500$ dado que su convergencia es más rápida y los efectos del parámetro c son distinguibles a menor horizonte. La comparación general entre algoritmos se realiza a $T = 1000$ para todos. Los rangos de hiperparámetros explorados han sido: $\varepsilon \in \{0, 0.01, 0.1, 0.2\}$ para ε -Greedy. $\lambda \in \{0.001, 0.01, 0.1\}$ y $\varepsilon_{\min} \in \{0, 0.1\}$ para ε -Decaimiento. $c \in \{0.1, 0.5, 1, 2, 5\}$ para UCB1. $\tau \in \{0.1, 0.5, 1, 2, 5\}$ para Softmax. La comparación final entre algoritmos usa $\varepsilon=0.1$, $\lambda=0.001/\varepsilon_{\min}=0.1$, $c=1$ y $\tau=1$ en los entornos Normal y Binomial. en Bernoulli se ajustan a $c=0.1$ y $\tau=0.1$ por el comportamiento diferencial observado en ese entorno.

Adicionalmente, se realiza un estudio específico sobre el efecto de la fase de inicialización, comparando ε -Greedy estándar frente a ε -Greedy con inicialización para $\varepsilon \in \{0, 0.01, 0.1\}$ sobre el entorno Normal ($T = 1000$, 500 ejecuciones). Las distribuciones de probabilidad han sido las siguientes:

- **Normal** $\mathcal{N}(\mu, \sigma^2)$: medias μ_i muestreadas uniformemente en $[1, 10]$ con $\sigma = 1.0$.
- **Bernoulli** $\text{Bern}(p)$: probabilidades p_i muestreadas en $[0.1, 0.9]$. Recompensas binarias $\{0, 1\}$.
- **Binomial** $\text{Bin}(n=10, p)$: probabilidades en $[0.1, 0.9]$, valor esperado $\mu_i = np_i \in [1, 9]$.

1.4.2. Métricas

Se emplean cuatro métricas complementarias para caracterizar el comportamiento de cada algoritmo:

1. **Recompensa promedio** ($\bar{r}_t$). Media de las recompensas obtenidas en el paso t sobre todos los episodios.
2. **Porcentaje de selecciones óptimas** (% opt.). Fracción de pasos en que el agente elige el brazo con mayor recompensa esperada, promediada sobre episodios.
3. **Arrepentimiento acumulado** (R_T): Es la suma de las recompensas perdidas por no elegir el brazo óptimo en cada paso. Se muestra además la cota de Lai-Robbins [2] ($\Theta(\ln T)$), una cota inferior asintótica que ningún algo-

ritmo consistente puede mejorar cuando $T \rightarrow \infty$. A horizonte finito el regret observado puede no haber alcanzado aún el régimen asintótico, por lo que los algoritmos pueden aparecer por debajo de la cota en los experimentos.

4. **Estadísticas de brazos.** Para cada algoritmo se visualiza, por brazo, la recompensa promedio observada y el número de selecciones acumuladas (indicado en la etiqueta del eje X). Las barras se colorean distinguiendo el brazo óptimo (verde), los subóptimos (azul) y los no visitados (gris), una línea discontinua roja marca el valor de recompensa del brazo óptimo como referencia.

1.4.3. Resultados y Análisis Crítico**Análisis de hiperparámetros** **ε -Greedy**

Distribución normal. Sin inicialización, $\varepsilon=0$ explota desde el primer paso el brazo seleccionado aleatoriamente al inicio, sin garantía de que sea el óptimo, generando regret elevado. $\varepsilon=0.01$ introduce una exploración muy escasa que no es suficiente para garantizar visitar el brazo óptimo con alta probabilidad, aunque mejora notablemente sobre $\varepsilon=0$. $\varepsilon=0.1$ equilibra exploración y explotación y obtiene el menor regret entre las configuraciones estudiadas. $\varepsilon=0.2$ introduce exploración excesiva, incluso tras identificar el brazo óptimo, el 20% de las tiradas se dedica a brazos subóptimos, elevando el regret respecto a $\varepsilon=0.1$.



Figura 1.1: Efecto de ε sobre el regret acumulado de ε -Greedy (distribución normal).

Distribución Bernoulli. Las recompensas binarias $\{0, 1\}$ reducen el rango absoluto de diferencias entre brazos ($\Delta_i = p^* - p_i \leq 0.8$), aunque el ranking de configuraciones se mantiene: $\varepsilon=0$ queda atrapado, $\varepsilon=0.01$ mejora ligeramente pero sigue explorando poco, $\varepsilon=0.1$ es el más eficiente y $\varepsilon=0.2$ genera exploración innecesaria. La diferencia entre $\varepsilon=0.1$ y $\varepsilon=0.2$ es más pequeña que en Normal porque los brazos subóptimos generan menos pérdida por tirada.

Distribución binomial. La binomial $\mathcal{B}(10, p)$ con $p_i \in [0.1, 0.9]$ produce valores esperados $\mu_i = 10p_i \in [1, 9]$, con un rango análogo al entorno Normal, esto hace que el patrón sea prácticamente idéntico: $\varepsilon=0$ falla gravemente, $\varepsilon=0.01$ explora insuficientemente, $\varepsilon=0.1$ es el óptimo y $\varepsilon=0.2$ introduce exploración residual que penaliza el largo plazo.



Figura 1.2: Efecto de ϵ sobre el regret acumulado de ϵ -Greedy (distribución Bernoulli).

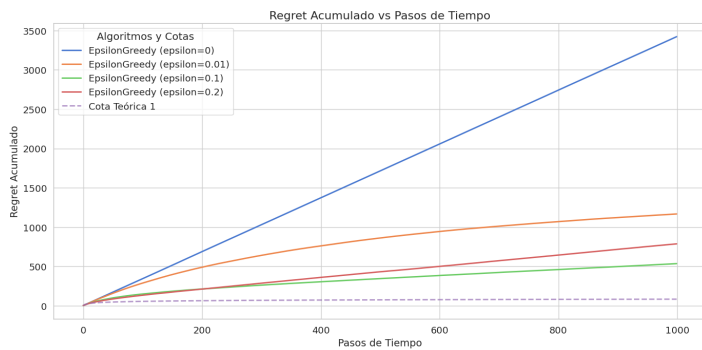


Figura 1.3: Efecto de ϵ sobre el regret acumulado de ϵ -Greedy (distribución binomial).

ϵ -Greedy con inicialización forzada

La fase de arranque cambia drásticamente el comportamiento para $\epsilon=0$. Sin inicialización (Figura 1.4a), el agente con $\epsilon=0$ selecciona el primer brazo visitado y lo explota de forma permanente, acumulando regret de manera practicamente lineal cuando ese brazo no es el óptimo. Con inicialización forzada (Figura 1.4b), $\epsilon=0$ se convierte en el algoritmo con menor regret, tras visitar los k brazos en la fase de arranque, identifica el óptimo con una estimación inicial y lo explota sin ninguna exploración residual posterior.

Para $\epsilon > 0$, la inicialización no elimina las visitas aleatorias de la fase de política, por lo que el ranking se invierte respecto al caso sin inicialización: $\epsilon=0 < \epsilon=0.01 < \epsilon=0.1$ en términos de regret. Esto ilustra que, una vez pagado el coste de la fase de arranque, cualquier exploración adicional es estrictamente perjudicial en un entorno estacionario.

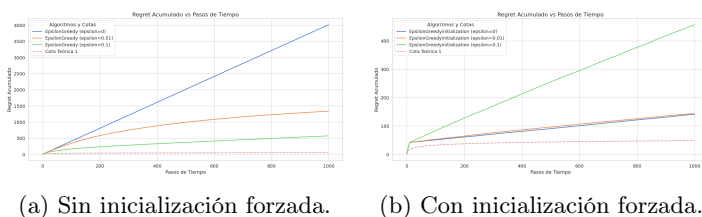


Figura 1.4: Comparación del regret acumulado de ϵ -Greedy con y sin inicialización forzada (distribución normal).

ϵ -Decaimiento

Distribución normal. λ controla la velocidad de descenso de ϵ_t desde $\epsilon_0=1.0$. Con $\lambda=0.1$, $\epsilon_t \approx 0.09$ en $t=100$ pasos, convergiendo rápidamente hacia la explotación, es la mejor configuración. Con $\lambda=0.01$, $\epsilon_t \approx 0.33$ en $t=300$, lo que hace que el decaimiento demasiado lento para $T=1000$ y el algoritmo sigue explorando considerablemente al final del episodio. Con $\lambda=0.001$, $\epsilon_t \approx 0.5$ incluso en $t=1000$, comportándose casi como un ϵ -greedy con ϵ fijo alto. Usar $\epsilon_{\min}=0.1$ mejora sobre $\epsilon_{\min}=0$ al mantener exploración residual que evita quedar atrapado definitivamente en brazos subóptimos si el brazo óptimo no fue bien identificado.

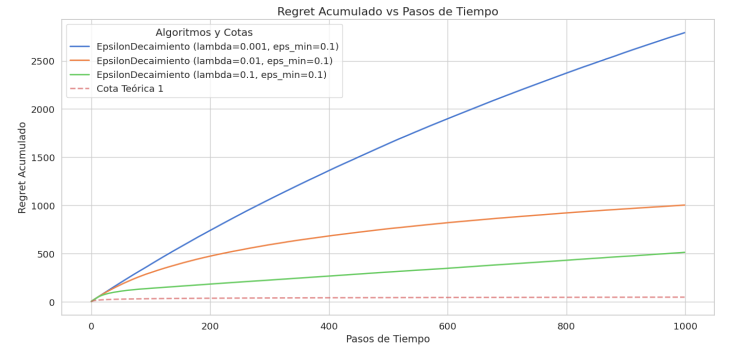


Figura 1.5: Efecto de λ sobre el regret acumulado de ϵ -Decaimiento (distribución normal).

Distribución Bernoulli. El ranking de λ se mantiene ($\lambda=0.1$ mejor). Con recompensas en $\{0, 1\}$ el decaimiento rápido es aún más ventajoso, las estimaciones $Q(a)$ convergen con menos muestras al estar acotadas, por lo que el algoritmo necesita menos exploración para identificar el brazo óptimo y beneficia más de converger pronto a la explotación.

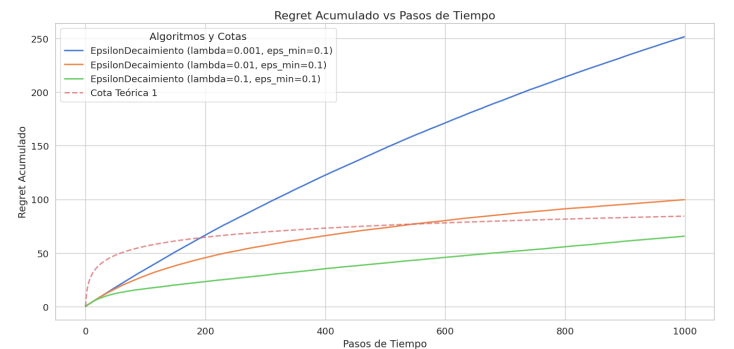


Figura 1.6: Efecto de λ sobre el regret acumulado de ϵ -Decaimiento (distribución Bernoulli).

Distribución binomial. Idéntico patrón al Normal, ya que la escala de recompensas $[1, 9]$ es comparable. $\lambda=0.1$ obtiene el menor regret en los tres entornos.

UCB1

Distribución normal. $c=1$ es el óptimo. Con $c=0.1$ el índice UCB1 está dominado por $Q(a)$, el algoritmo se comporta casi como greedy y puede quedar atrapado en un subóptimo local por explorar insuficientemente los brazos poco visitados. Con $c=5$ el bonus domina completamente, forzando la visita repetida a brazos ya bien estimados, el algoritmo dedica la mayor

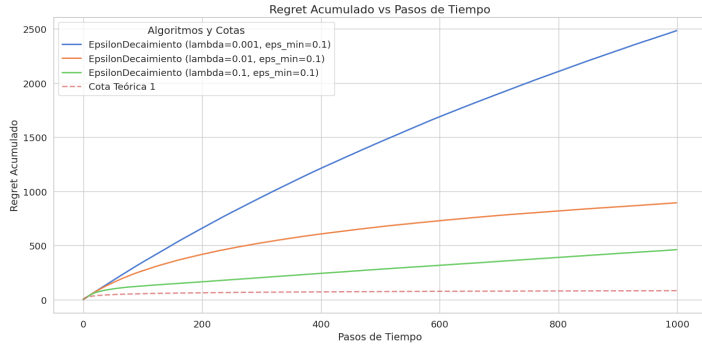


Figura 1.7: Efecto de λ sobre el regret acumulado de ε -Decaimiento (distribución binomial).

parte del presupuesto a exploración innecesaria, elevando el regret. Los valores intermedios $c=0.5$ y $c=2$ confirman que $c=1$ calibra el balance correctamente para recompensas en el rango $[1, 10]$.



Figura 1.8: Efecto de c sobre el regret acumulado de UCB1 (distribución normal).

Distribución Bernoulli. El óptimo cambia a $c=0.1$. Las diferencias absolutas entre brazos $\Delta_i = p^* - p_i \leq 0.8$ son mucho menores que en Normal ($\Delta_i \leq 9$). Con $c=1$ el bono sobrepondera la incertidumbre relativa y lleva a explorar excesivamente brazos que ya han sido suficientemente identificados como subóptimos. Con $c=0.1$ el índice está calibrado a la escala de las recompensas binarias y dirige la exploración de forma más eficiente.



Figura 1.9: Efecto de c sobre el regret acumulado de UCB1 (distribución Bernoulli).

Distribución binomial. Como en Normal, $c=1$ es el óptimo, ya que la escala de recompensas $[1, 9]$ es comparable y el mismo nivel de bono resulta adecuado.



Figura 1.10: Efecto de c sobre el regret acumulado de UCB1 (distribución binomial).

Softmax

Distribución normal. $\tau=1$ es el óptimo para recompensas en el rango $[1, 10]$. Con $\tau=0.1$ la distribución de selección está muy concentrada, el brazo con mayor $Q(a)$ recibe casi toda la probabilidad ($>99.9\%$ cuando las diferencias de valor superan la unidad), lo que equivale a una política casi greedy que puede quedar atrapada en subóptimos locales. Con $\tau=5$ las probabilidades se aplanan y la selección es casi uniforme sobre los k brazos, ignorando la información acumulada en $Q(a)$ y generando el mayor regret. $\tau=1$ equilibra ambos extremos.



Figura 1.11: Efecto de τ sobre el regret acumulado de Softmax (distribución normal).

Distribución Bernoulli. El óptimo cambia a $\tau=0.1$. Las diferencias absolutas entre $Q(a)$ están acotadas en $[0, 1]$, por lo que con $\tau=1$, el cociente $Q(a)/\tau$ produce exponentes similares para todos los brazos, aplanando la distribución hacia la selección casi uniforme. Con $\tau=0.1$ los exponentes amplían las diferencias relativas y concentran la probabilidad adecuadamente en los brazos con mayor $Q(a)$.

Distribución binomial. Como en Normal, $\tau=1$ es el óptimo, ya que la escala de recompensas $[1, 9]$ produce diferencias $Q(a)$ de orden comparable al entorno Normal.

Comparación general

La comparación general incluye cuatro algoritmos: ε -Greedy, ε -Decaimiento, UCB1 y Softmax. El ε -Greedy con inicialización se excluye de esta comparativa porque su ventaja sobre ε -Greedy estándar se concentra exclusivamente en $\varepsilon=0$, que no tiene interés en la comparativa general al ser la política puramente greedy. Para $\varepsilon > 0$, la fase de arranque apenas altera el comportamiento a largo plazo respecto a ε -Greedy

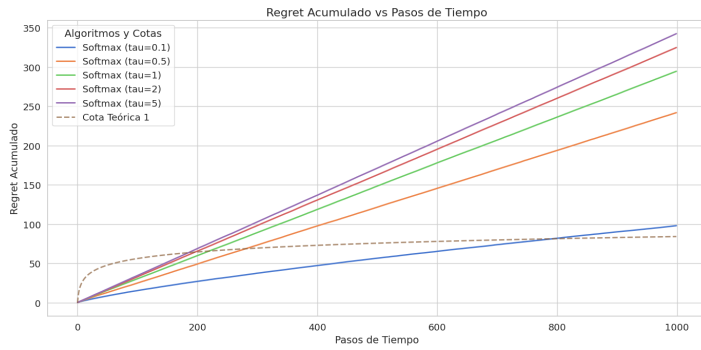


Figura 1.12: Efecto de τ sobre el regret acumulado de Softmax (distribución Bernoulli).

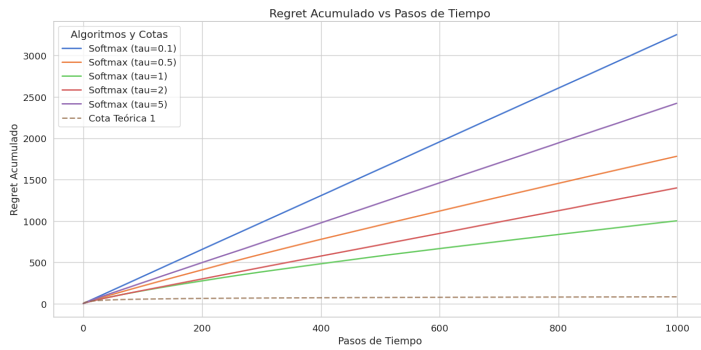


Figura 1.13: Efecto de τ sobre el regret acumulado de Softmax (distribución binomial).

estándar, como se evidencia en el estudio aislado de la sección anterior.

Distribución normal

La Figura 1.14 muestra que UCB1 ($c=1$) es el único algoritmo cuyo regret crece de forma claramente sublineal y se aproxima a la pendiente de la cota de Lai-Robbins. ε -Decaimiento ($\lambda=0.1$) supera a ε -Greedy ($\varepsilon=0.1$) porque concentra la exploración al principio y reduce ε_t progresivamente, evitando el coste constante de mantener una fracción fija de tiradas aleatorias. Softmax ($\tau=1$) es el peor, con recompensas en $[1, 10]$, los cocientes $Q(a)/\tau$ producen diferencias de exponente ~ 9 , pero el mecanismo de selección proporcional nunca converge a greedy y dedica una fracción persistente de tiradas a brazos subóptimos.



Figura 1.14: Regret acumulado en distribución normal para la mejor configuración de cada algoritmo.

La Figura 1.15 confirma este ranking desde la perspectiva

del porcentaje de selección óptima. UCB1 alcanza el mayor porcentaje de selecciones óptimas y lo hace de forma estable desde las primeras decenas de pasos, gracias a que el bono dirige la exploración hacia los brazos con mayor incertidumbre, descartando los subóptimos a medida que se acumula evidencia. ε -Greedy también porcentajes altos pero de forma más progresiva, es bajo durante la fase de arranque y se dispara una vez que empieza a explotar. Los métodos con ε constante o decaimiento más lento oscilan alrededor de un valor intermedio, reflejo de que siguen dedicando tiradas a brazos subóptimos incluso cuando el óptimo ya ha sido identificado. Por otro lado, vemos como Softmax es el que peor porcentaje obtiene quedándose en un 40 % frente al resto que supera el 80 %.

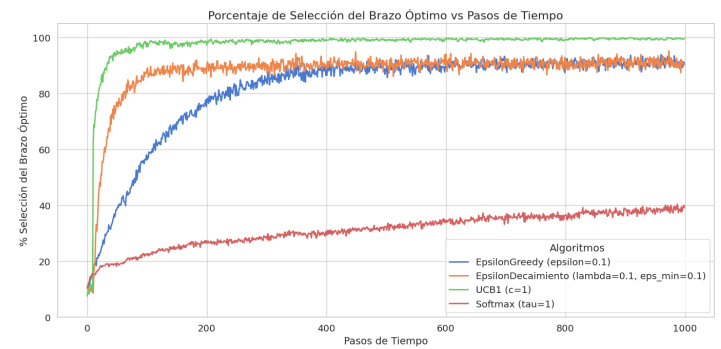


Figura 1.15: Porcentaje de selección del brazo óptimo en distribución normal para la mejor configuración de cada algoritmo.

La Figura 1.16 muestra la recompensa promedio por paso, que es el complemento directo del regret, los algoritmos con menor regret obtienen mayor recompensa promedio. UCB1 converge antes a la recompensa del brazo óptimo, seguido por ε -Greedy con decaimiento y ε -Greedy, mientras que Softmax se estabiliza por debajo al dedicar siempre cierta probabilidad a brazos de menor valor.

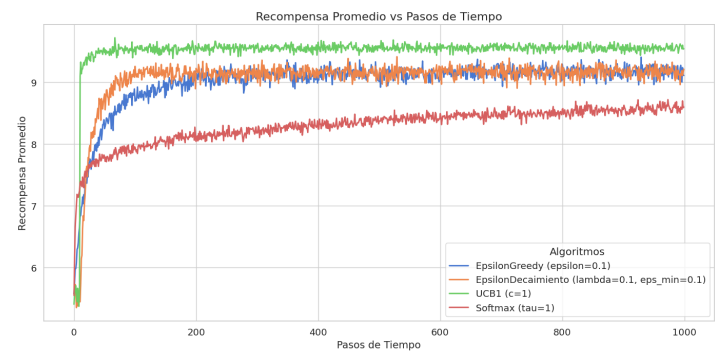


Figura 1.16: Recompensa promedio por paso en distribución normal para la mejor configuración de cada algoritmo.

Distribución binomial

La distribución binomial $\mathcal{B}(10, p)$ con $p_i \in [0.1, 0.9]$ reproduce el patrón de la distribución normal, ya que la escala de recompensas $[1, 9]$ es comparable. Como muestra la Figura 1.17, UCB1 ($c=1$) vuelve a dominar claramente con el menor regret acumulado. El resto de algoritmos se sitúan por encima de la cota de Lai-Robbins a $T \leq 1000$, excepto UCB1 que está un poco por debajo, aun así esto no contradice la teoría, sino que

refleja que la cota es asintótica y, a horizonte finito, el algoritmo aun no ha alcanzado dicho límite inferior. ε -Decaimiento supera ligeramente a ε -Greedy y ambos mejoran a Softmax.

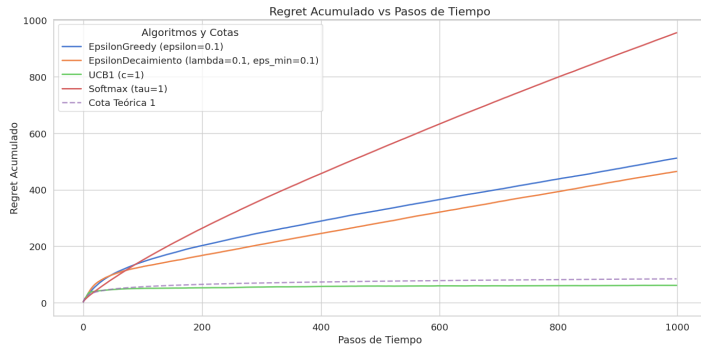


Figura 1.17: Regret acumulado en distribución binomial para la mejor configuración de cada algoritmo.

La Figura 1.18 muestra que UCB1 alcanza el porcentaje de selección óptima más alto y más rápidamente, confirmando que su bono de exploración está bien calibrado para esta escala de recompensas. El resto de algoritmos convergen a porcentajes menores debido a que siguen dedicando tiradas a brazos subóptimos de forma sistemática (ε fijo) o proporcional (Softmax). El perfil temporal es análogo al caso normal, con la diferencia de que las curvas convergen algo más lentamente al ser las recompensas enteras y la varianza de las estimaciones $Q(a)$ algo mayor.

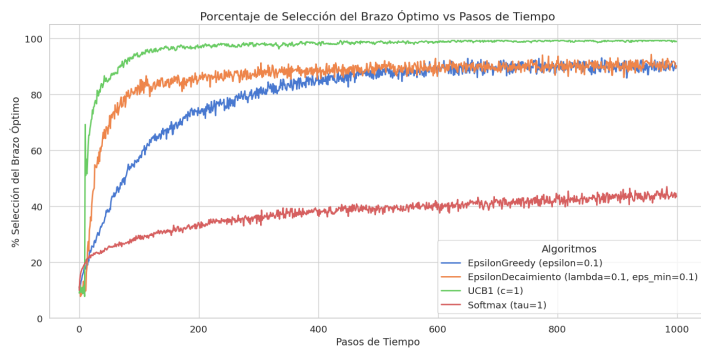


Figura 1.18: Porcentaje de selección del brazo óptimo en distribución binomial para la mejor configuración de cada algoritmo.

La Figura 1.19 corrobora los resultados anteriores, UCB1 alcanza antes la recompensa del brazo óptimo y la mantiene más estable, mientras que Softmax se sitúa en el extremo inferior por la misma razón que en Normal.

Distribución Bernoulli

La distribución Bernoulli introduce el cambio cualitativo más relevante del estudio. Como se aprecia en la Figura 1.20, los hiperparámetros óptimos de UCB1 y Softmax se desplazan respecto a Normal y Binomial. Ahora, UCB1 necesita $c=0.1$ y Softmax $\tau=0.1$, porque las recompensas binarias $\{0, 1\}$ reducen las diferencias absolutas entre brazos, como ya vimos anteriormente al analizar los resultados de los algoritmos con distintos hiperparámetros. Con estos parámetros ajustados, UCB1 vuelve a obtener el menor regret y se sitúa muy por debajo de la cota de Lai-Robbins, lo que, como ya se ha seña-

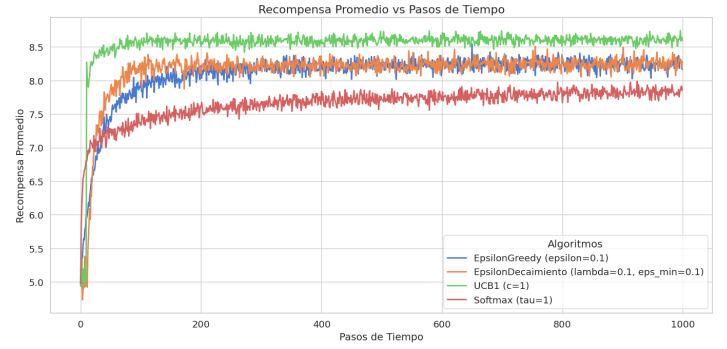


Figura 1.19: Recompensa promedio por paso en distribución binomial para la mejor configuración de cada algoritmo.

lado, es consistente con el carácter asintótico de dicha cota. ε -Greedy y ε -Decaimiento se aproximan más a la cota que en los otros entornos, ya que la exploración aleatoria resulta relativamente menos costosa cuando las recompensas son binarias. Softmax con $\tau=0.1$ mejora su posición relativa respecto a los entornos anteriores, aun así esto se debe más a la escala de recompensas, ya que como se observa sigue siendo el que mayor regret obtiene.

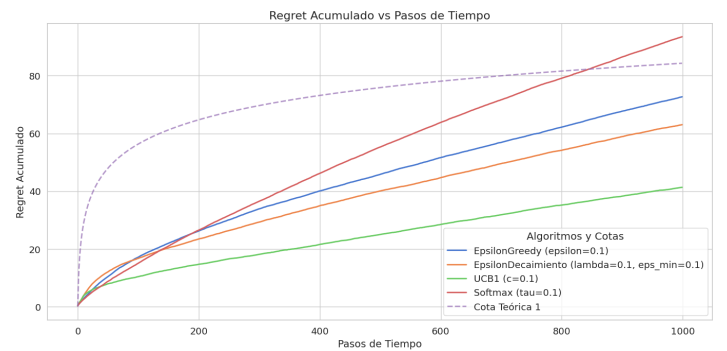


Figura 1.20: Regret acumulado en distribución Bernoulli para la mejor configuración de cada algoritmo.

El porcentaje de selección óptima (Figura 1.21) es el resultado más llamativo de la distribución Bernoulli, a diferencia de Normal y Binomial, UCB1 *no* es el algoritmo que mayor porcentaje de selección óptima alcanza. Las recompensas binarias $\{0, 1\}$ introducen una varianza muy elevada en las estimaciones $Q(a)$, lo que hace que el bono con $c=0.1$ sea insuficiente para distinguir rápidamente el brazo óptimo de los subóptimos próximos, y el algoritmo sigue explorando otros brazos durante más tiempo.

La recompensa promedio (Figura 1.22) está acotada en $[0, 1]$, lo que comprime las diferencias absolutas entre algoritmos respecto a los entornos anteriores, y la convergencia a la recompensa del brazo óptimo parece ser más lenta que en Normal y Binomial, consecuencia directa de la mayor varianza de las estimaciones con recompensas Bernoulli. Además, toda la distribución es más ruidosa y las curvas de los distintos algoritmos están más juntas entre sí que en los entornos anteriores, esto debido a que las recompensas binarias provocan que los algoritmos cambien de brazo con mayor frecuencia durante más tiempo y el coste de equivocarse sea pequeño, lo que reduce las diferencias observables entre estrategias. En conjunto, estos resultados confirman el ranking general a tra-

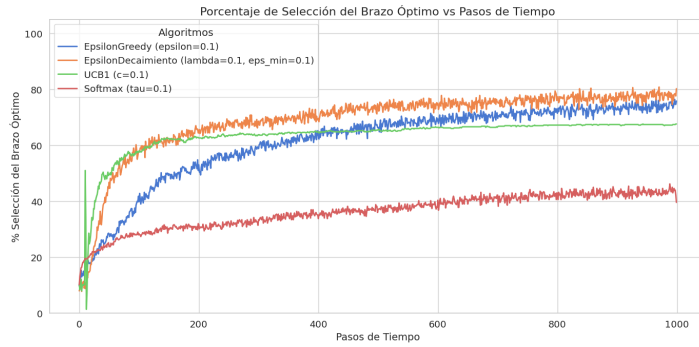


Figura 1.21: Porcentaje de selección del brazo óptimo en distribución Bernoulli para la mejor configuración de cada algoritmo.

vés de las tres distribuciones. UCB1 obtiene el menor regret en los tres entornos, seguido de ϵ -Decaimiento, ϵ -Greedy, y Softmax, que cierra siempre con el mayor regret acumulado.

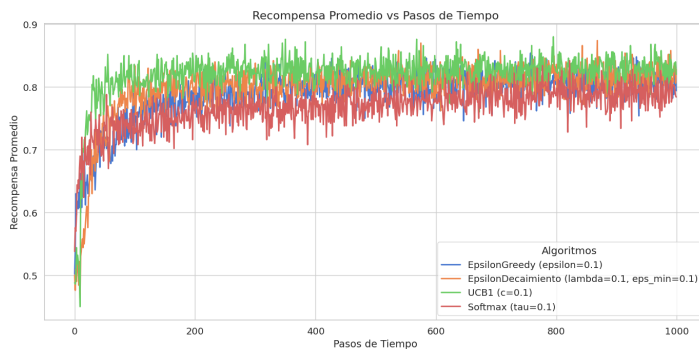


Figura 1.22: Recompensa promedio por paso en distribución Bernoulli para la mejor configuración de cada algoritmo.

1.5. Conclusiones

Los experimentos confirman que UCB1 es el algoritmo con menor regret acumulado en las tres distribuciones estudiadas, gracias a su bono que dirige la exploración de forma adaptativa hacia los brazos con mayor incertidumbre. ϵ -Decaimiento supera a ϵ -Greedy al concentrar la exploración al inicio y converger hacia la explotación, evitando el coste constante de mantener una fracción fija de tiradas aleatorias. Softmax es el peor algoritmo evaluado, su exploración proporcional nunca decrece y es el más sensible al hiperparámetro τ , siendo el más difícil de ajustar. Un hallazgo transversal es que no existe una configuración universal de hiperparámetro, los valores óptimos de c y τ se desplazan significativamente entre Normal/Binomial y Bernoulli, porque la escala de las recompensas determina la magnitud de las diferencias entre brazos y, por tanto, la intensidad de exploración necesaria.

Durante los experimentos también nos hemos dado cuenta de que la mejor métrica ha sido el regret acumulado. La recompensa promedio y el porcentaje de selección óptima, aunque intuitivas, presentan limitaciones. Ña recompensa promedio depende de la escala de la distribución y no permite comparar resultados entre entornos distintos (una recompensa de 8,0 en Normal no es equivalente a 0,8 en Bernoulli), mientras que el porcentaje de selección óptima puede resultar engañoso cuando varios brazos tienen medias muy próximas, ya que el

algoritmo puede seleccionar frecuentemente un brazo subóptimo casi tan bueno como el óptimo y obtener un regret bajo pero un porcentaje de selección óptima aparentemente pobre. El regret acumulado, en cambio, captura directamente el coste de no haber explotado siempre el brazo óptimo, es comparable entre distribuciones una vez normalizado, y cuenta con respaldo teórico sólido a través de la cota de Lai-Robbins, que proporciona un punto de referencia absoluto independiente del algoritmo. Por estas razones, el regret es la métrica más informativa y la que mejor refleja la eficiencia global de cada política.

Reflexión sobre el impacto en el campo

El problema del bandido de k -brazos es el punto de partida fundamental del aprendizaje por refuerzo, ya que captura en su forma más pura el dilema entre exploración y explotación, el agente debe decidir si confiar en el conocimiento acumulado o invertir en reducir su incertidumbre sobre las alternativas no exploradas. Este dilema está presente en todos los problemas de RL más complejos, desde la navegación en entornos tabulares hasta el ajuste fino de modelos de lenguaje. Estudiar los mecanismos de exploración en el bandido permite entender y diseñar mejor las estrategias de exploración en entornos secuenciales con estado, donde los mismos principios (bonus de incertidumbre, decaimiento de exploración, selección proporcional al valor) se trasladan directamente a métodos como Q-learning con exploración UCB o políticas softmax en actor-critic.

Limitaciones

En cuanto a las limitaciones del estudio, los experimentos se limitan a $k=10$ brazos y horizonte $T \leq 1000$ (excepto para UCB1, con horizonte $T \leq 500$) con un único conjunto de parámetros por distribución. Este horizonte puede ser demasiado corto para que la cota de Lai-Robbins se manifieste en todos los algoritmos, y los resultados dependen en cierta medida de la instancia concreta de brazos generada. Además, el entorno es estrictamente estacionario, lo que favorece a algoritmos como ϵ -Greedy con inicialización forzada o UCB1, que asumen que las distribuciones de recompensa no cambian a lo largo del tiempo.

Líneas de Trabajo Futuro

Como líneas futuras, sería interesante ampliar los experimentos a horizontes mayores y promediar sobre múltiples instancias aleatorias de brazos para obtener resultados estadísticamente más robustos. Por otro lado, introducir decaimiento en τ para Softmax podría mejorar notablemente su rendimiento acercándolo a ϵ -Decaimiento. También cabría explorar algoritmos más avanzados como UCB2, KL-UCB o Thompson Sampling, que incorporan la estructura de la distribución de recompensas para calibrar el bonus de exploración de forma más precisa, y cuya comparación empírica con UCB1 en los tres entornos estudiados sería de alto interés.

Capítulo 2

Aprendizaje en Entornos Complejos

2.1. Introducción

El aprendizaje por refuerzo representa una de las ramas más diversas de la inteligencia artificial, empleada para derrotar a campeones del mundo en juegos de mesa estratégicos o en la construcción de modelos de lenguaje avanzados. A diferencia del problema del bandido de k -brazos, donde las decisiones se toman en un entorno estático, los problemas del mundo real se desarrollan en un contexto dinámico. Cada decisión tomada por el agente modifica el estado del entorno, lo que requiere cálculos constantes para actualizar su percepción del mundo.

La motivación de este trabajo radica en comprender y aplicar algoritmos capaces de resolver tareas sin un conocimiento previo de la dinámica del entorno. Se pretende evolucionar desde los métodos tabulares clásicos, viables en espacios de estados reducidos, hasta el control con aproximaciones de función mediante redes neuronales, necesario para abordar entornos continuos y complejos.

Los objetivos principales de este informe son:

1. Implementar y comparar el rendimiento de algoritmos de control tabular (Monte Carlo, SARSA y Q-Learning) en entornos de estados discretos.
2. Desarrollar y evaluar algoritmos de control aproximado (Deep Q-Learning y SARSA semi-gradiente) en entornos continuos.
3. Analizar el comportamiento, el rendimiento y la convergencia de los agentes basándose en la librería de gymnasium [9] para llevar a cabo las pruebas.

El documento se organiza de la siguiente manera: la sección de Desarrollo detalla el marco teórico necesario para aplicar los algoritmos de aprendizaje. La sección de Algoritmos expone el pseudocódigo y la lógica de los métodos implementados. Posteriormente, la Evaluación presenta los resultados empíricos obtenidos en los entornos seleccionados, y finalmente, se extraen las Conclusiones y líneas de mejora futuras.

2.2. Desarrollo

2.2.1. Definición Formal del Problema

El aprendizaje en entornos complejos se formaliza a través de un Proceso de Decisión de Markov (MDP). Un MDP se define mediante la tupla $\langle \mathcal{S}, \mathcal{A}, \{\mathcal{P}^{-1}\}_{-}, \mathcal{R}, \gamma \rangle$, donde en cada instante t , el agente observa un estado $S_t \in \mathcal{S}$, ejecuta una

acción $A_t \in \mathcal{A}$ siguiendo una política $\pi(a|s)$, y transita a un nuevo estado S_{t+1} recibiendo una recompensa R_{t+1} .

Definición 2.2.1 (Política). Una política, representada como π , es una función que indica qué acción realizar de acuerdo al estado actual del entorno. Puede ser determinista, asignando una acción fija a un estado, o estocástica, proporcionando una distribución de probabilidad sobre las acciones posibles.

El objetivo del agente es encontrar la política óptima π^* que maximice el retorno esperado (la suma de recompensas descontadas a futuro):

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.1)$$

Para poder escoger la acción que lleve a un estado donde la recompensa sea máxima, el agente construye una representación interna del entorno basado en sus experiencias anteriores. La puntuación que asigna a cada estado el agente se basa en las ecuaciones de Bellman, que según el algoritmo implementado, se modifica para poder aplicar distintos enfoques. La fórmula general es la siguiente

$$v(s) = \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s] \quad (2.2)$$

Esta fórmula desglosa el valor de un estado en dos componentes principales:

- R_{t+1} : Es la recompensa inmediata que el agente recibe al realizar una transición desde el estado actual s .
- $\gamma v(S_{t+1})$: Representa el valor del siguiente estado (S_{t+1}) multiplicado por el factor de descuento γ . Este factor determina la importancia de las recompensas futuras frente a las inmediatas.

Esta ecuación representa que el valor de un estado es la recompensa inmediata más un valor a futuro, que representa el pasar a un próximo estado.

Para aplicar esta ecuación a cierto problema, se necesitan datos que involucren la obtención de recompensas.

Definición 2.2.2 (Episodio). Un episodio se define como una secuencia que involucra pasar por un estado, tomar una acción y obtener una recompensa por alcanzar otro estado, terminando el episodio en un estado terminal teóricamente. En tareas

episódicas, el agente utiliza la retroalimentación obtenida al final o durante el transcurso del episodio para maximizar la recompensa acumulativa futura

2.2.2. Enfoques y Métodos Utilizados

Cuando no se conoce el modelo de transiciones $\mathcal{P}$ ni la función de recompensa $\mathcal{R}$, el agente debe aprender a través de la experiencia. El objetivo que tiene este es actualizar su representación del entorno (tabla de estados-acciones Q) para poder elegir, dado un estado S , la acción A que maximice su retorno. Las familias de métodos estudiadas se muestran a continuación por antigüedad.

- ▶ **Métodos de Monte Carlo (MC):** Requieren que el episodio termine para calcular el retorno real G_t y actualizar el valor de los estados visitados. Son métodos de alta varianza pero sin sesgo.
- ▶ **Métodos de Diferencias Temporales (TD):** Combinan ideas de MC y Programación Dinámica. Actualizan las estimaciones basándose en otras estimaciones parciales (bootstrapping) paso a paso, sin esperar al final del episodio.
- ▶ **Control con Aproximaciones:** Cuando el espacio de estados es inmenso o continuo, es imposible mantener una tabla $Q(s, a)$. Se opta por aproximar la función de valor como una forma funcional parametrizada con un vector de pesos.

Estos enfoques mejoran gradualmente la aplicabilidad del agente. Los métodos TD mejoran a MC en entornos largos o continuos al aprender paso a paso, mientras que los métodos aproximados (DQN) suponen el salto cualitativo definitivo para resolver tareas complejas inabarcables mediante técnicas tabulares.

Aunque los últimos métodos mencionados sean los más complejos, en [4] se utilizó Q-Learning con ciertas optimizaciones para la optimización del movimiento de aeronaves en tierra dentro de los aeropuertos, demostrando ser más eficiente que los métodos tradicionales basados en reglas al adaptarse dinámicamente a las condiciones del aeropuerto. Otro ejemplo conocido que usaba métodos aproximados es el de [3], donde se usó Deep Q-Learning para conseguir resultados propios del estado del arte en cuanto a rendimiento en juegos de la consola Atari 2600, introduciendo únicamente píxeles como input a la red, obteniendo rendimientos parecidos a los de un humano. Este trabajo sentó las bases del control aproximado moderno, y fue posteriormente mejorado por [10], que redujo la sobreestimación de Q-values mediante el desacoplamiento de la selección y evaluación de acciones.

Este trabajo se diferencia de otros enfoques previos al alejarse de la mera réplica de tutoriales para establecer y seguir un criterio de evaluación y una metodología propia. En concreto, cubre el espectro completo de métodos, desde control tabular (Monte Carlo, SARSA, Q-Learning) hasta control aproximado (SARSA semi-gradiente, DQN), con la transición entre ambos motivada formalmente por las dimensiones del espacio de observación de cada entorno. Además, se aplica un marco de evaluación uniforme y reproducible que permite compara-

ciones directas entre algoritmos bajo las mismas condiciones experimentales.

2.3. Algoritmos

A continuación, se describen los algoritmos estudiados. Todos los agentes siguen la misma interfaz estructural basada en los métodos `get_action(obs)` y `update(...)` típica de gymnasium.

Monte Carlo Control (Off-policy)

Este algoritmo separa la política que se está aprendiendo (objetivo, π) de la política que genera el comportamiento (behavior, b), permitiendo aprender una política determinista óptima a partir de datos generados por una política exploratoria ε .

El episodio completo se genera con b y luego se recorre de atrás hacia adelante acumulando el retorno $G \leftarrow \gamma G + R_{t+1}$. El sesgo introducido por la diferencia entre políticas se corrige mediante muestreo por importancia ponderado, donde el peso acumulado W representa el producto $\prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$. Dado que π es determinista, $\pi(A_t|S_t) = 1$ solo si $A_t = \pi(S_t)$, y cero en caso contrario. Por ello, en cuanto la acción tomada no coincide con la acción greedy, cualquier peso posterior sería nulo y se abandona el recorrido del episodio, ahorrando cómputo. La actualización de Q usa la media ponderada acumulada $C(s, a)$ en lugar de un conteo simple, garantizando estimaciones consistentes con pesos variables.

Algorithm 7 Control de Monte Carlo (off-policy)

- 1: **Inicializar**, para todo $s \in \mathcal{S}, a \in \mathcal{A}(s)$:
 - 2: $Q(s, a) \in \mathbb{R}$ (arbitrariamente)
 - 3: $C(s, a) \leftarrow 0$
 - 4: $\pi(s) \leftarrow \arg \max_a Q(s, a)$ ▷ con empates rotos consistentemente
 - 5: **Repetir para siempre (para cada episodio):**
 - 6: $b \leftarrow$ cualquier política
 - 7: Generar un episodio usando b :
 $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$
 - 8: $G \leftarrow 0$
 - 9: $W \leftarrow 1$
 - 10: **for** $t = T - 1, T - 2, \dots, 0$ **do**
 - 11: $G \leftarrow \gamma G + R_{t+1}$
 - 12: $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
 - 13: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$
 - 14: $\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$ ▷ con empates rotos consistentemente
 - 15: **if** $A_t \neq \pi(S_t)$ **then**
 - 16: **salir del bucle interno** ▷ proceder al siguiente episodio
 - 17: **end if**
 - 18: $W \leftarrow W \frac{1}{b(A_t|S_t)}$
 - 19: **end for**
-

A pesar de que en la práctica los algoritmos de diferencias temporales suelen ser más eficientes, el concepto de Monte Carlo se aplica en algoritmos más complejos como REINFORCE, que se basa en el gradiente de política. Se elige este

método para el entorno Taxi-v3, el cual luego se explicará, por su capacidad de aprender una política óptima determinista sin necesidad de explorar directamente con ella, lo que resulta ventajoso en un entorno con penalizaciones explícitas por acciones ilegales (-10), donde una política exploratoria directa incurriría en mayor coste durante el entrenamiento.

Monte Carlo Control Todas las Visitas (On-policy)

En este caso la política de comportamiento y la política objetivo son la misma (ϵ -greedy), eliminando la necesidad de muestreo por importancia. A diferencia del criterio de primera visita, se actualiza $Q(s, a)$ **cada vez** que el par (s, a) aparece en el episodio, usando media incremental.

Tras completar el episodio, el retorno se acumula retrocediendo desde el estado terminal, en cada paso t se calcula $G \leftarrow \gamma G + R_{t+1}$ y se actualiza Q con la fórmula $Q \leftarrow Q + \frac{1}{N(S_t, A_t)}[G - Q(S_t, A_t)]$, donde N cuenta las visitas al par en todos los episodios. El criterio de todas las visitas aumenta el número de actualizaciones por episodio respecto a primera visita, acelerando la convergencia a costa de introducir una ligera correlación entre las estimaciones de un mismo par dentro del mismo episodio.

Algorithm 8 MC Control On-policy, todas las visitas

Require: $\epsilon > 0$

```

1: Inicializar:  $Q(s, a) \in \mathbb{R}$  (arbitrariamente),  $N(s, a) \leftarrow 0$ ,  

    $\pi \leftarrow \epsilon$ -greedy respecto a  $Q$ 
2: Repetir para siempre (para cada episodio):
3:   Generar un episodio usando  $\pi$ :  

    $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$ 
4:    $G \leftarrow 0$ 
5:   for  $t = T - 1, T - 2, \dots, 0$  do
6:      $G \leftarrow \gamma G + R_{t+1}$ 
7:      $N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$ 
8:      $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)}[G - Q(S_t, A_t)]$ 
9:      $\pi(S_t) \leftarrow \epsilon$ -greedy respecto a  $Q(S_t, \cdot)$ 
10:  end for
```

Se incluye este método junto al off-policy en Taxi-v3 para analizar el compromiso entre ambos enfoques, al mantener una única política ϵ -greedy, el aprendizaje es más directo pero la política nunca converge a una solución completamente greedy, pues requiere mantener $\epsilon > 0$ para garantizar exploración continua.

Sarsa (on-policy TD control)

A diferencia de los métodos de Monte Carlo, Sarsa es un algoritmo de Diferencias Temporales (TD) que utiliza la técnica de *bootstrapping*. Esto le permite realizar actualizaciones paso a paso (TD(0)) basándose en estimaciones parciales del entorno, lo que resulta especialmente eficiente en tareas largas o continuas donde no siempre es posible esperar al estado terminal.

La característica distintiva de Sarsa es su naturaleza **on-policy**. Sarsa actualiza el valor $Q(S, A)$ utilizando la acción real A' que el agente ya ha seleccionado para el siguiente es-

Algorithm 9 Sarsa: Control TD en la política (on-policy) para estimar $Q \approx q_*$

Require: Parámetros del algoritmo: tamaño de paso $\alpha \in (0, 1]$, $\epsilon > 0$ pequeño

```

1: Inicializar  $Q(s, a)$ , para todo  $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$ , arbitrariamente excepto que  $Q(\text{terminal}, \cdot) = 0$ 
2: for cada episodio do
3:   Inicializar  $S$ 
4:   Elegir  $A$  desde  $S$  usando una política derivada de  $Q$  (p.e.,  $\epsilon$ -greedy)
5:   while  $S$  no sea terminal do
6:     Realizar la acción  $A$ , observar  $R$  y el siguiente estado  $S'$ 
7:     Elegir  $A'$  desde  $S'$  usando una política derivada de  $Q$  (p.e.,  $\epsilon$ -greedy)
8:      $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$ 
9:      $S \leftarrow S'$ 
10:     $A \leftarrow A'$ 
11:   end while
12: end for
```

tado. Este enfoque obliga al agente a aprender el valor de la política que está ejecutando en ese momento, incluyendo el ruido.^o el riesgo derivado de la exploración aleatoria (ϵ).

La actualización de los valores Q se fundamenta en la ecuación de Bellman, la cual define el valor de un estado en base a la recompensa inmediata y el valor futuro descontado, $R + \gamma Q(S', A')$, que al compararse mediante resta con la estimación o creencia previa del agente, $Q(S, A)$ o $\hat{q}(S, A, \mathbf{w})$, genera el denominado error de Diferencia Temporal (error TD). Este error permite realizar ajustes incrementales que refinan el conocimiento del agente y guían la convergencia de la función de valor hacia la política óptima.

Como consecuencia de este aprendizaje sobre la política real, Sarsa se convierte en un algoritmo más conservador en comparación con otros algoritmos off-policy, que toma en cuenta sus decisiones en el futuro.

Q-Learning (off-policy TD control)

Q-Learning es un método off-policy de diferencias temporales que aprende directamente la función de valor-acción óptima q_* . En cada paso, tras ejecutar la acción A en el estado S y observar la recompensa R y el estado siguiente S' , actualiza Q con la regla:

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$$

El término entre corchetes es el *error de diferencia temporal*, la diferencia entre el retorno estimado asumiendo comportamiento óptimo desde S' y el valor actual $Q(S, A)$. La clave off-policy reside en que el objetivo siempre usa $\max_a Q(S', a)$, es decir, evalúa la política greedy independientemente de la acción que el agente realmente tomará en S' (que puede ser exploratoria). Esto separa la política de comportamiento (ϵ -greedy) de la política objetivo (greedy), y garantiza convergencia a q_* bajo condiciones estándar de visita y decrecimiento del paso α .

Su comportamiento contrasta directamente con SARSA en

entornos con zonas peligrosas, Q-Learning aprende el camino óptimo greedy (más corto, bordeando el peligro), mientras que SARSA, al evaluarse con la acción que realmente tomará, aprende una política más conservadora que evita zonas de alto riesgo por la posibilidad de ejecutar acciones exploratorias en su proximidad.

Algorithm 10 Q-learning: Control TD fuera de la política (off-policy) para estimar $\pi \approx \pi_*$

Require: Parámetros del algoritmo: tamaño de paso $\alpha \in (0, 1]$, $\varepsilon > 0$ pequeño

- 1: Inicializar $Q(s, a)$, para todo $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrariamente excepto que $Q(\text{terminal}, \cdot) = 0$
- 2: **for** cada episodio **do**
- 3: Inicializar S
- 4: **repeat** ▷ Para cada paso del episodio
- 5: Elegir A desde S usando una política derivada de Q (p.e., ε -greedy)
- 6: Realizar la acción A , observar R y el siguiente estado S'
- 7: $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- 8: $S \leftarrow S'$
- 9: **until** S es terminal
- 10: **end for**

Q-Learning, al ser off-policy, se aproxima más a la política óptima que SARSA debido a que su política objetivo es greedy y siempre escoge el estado con máximo valor.

Sarsa semi-gradiente episódico

El algoritmo Sarsa semi-gradiente representa el paso de los métodos tabulares y sus tablas de valores discretas hacia la generalización en espacios de estados continuos o de alta dimensionalidad. Mientras que en los métodos tabulares cada par (s, a) se trata de forma aislada, este enfoque utiliza una función de aproximación $\hat{q}(s, a, \mathbf{w})$ parametrizada por un vector de pesos $\mathbf{w} \in \mathbb{R}^d$

El semi-gradiente es fundamental en esta técnica. En un descenso de gradiente estándar, se consideraría el gradiente de todo el error cuadrático respecto a los pesos. Sin embargo, en Sarsa, el valor objetivo (target) $R + \gamma \hat{q}(S', A', \mathbf{w})$ depende también de los pesos $\mathbf{w}$. Para simplificar el cálculo y permitir el *bootstrapping*, el algoritmo ignora la dependencia del *target* respecto a $\mathbf{w}$ y solo calcula el gradiente del término que se está intentando ajustar: $\nabla \hat{q}(S, A, \mathbf{w})$

Esta técnica es la que permite utilizar redes neuronales como aproximadores de función, donde el vector $\mathbf{w}$ representa los parámetros de la red y el gradiente se calcula mediante retropropagación (backpropagation).

Deep Q-Learning (DQN) con Experience Replay

DQN extiende Q-Learning clásico [7] para superar el problema de la dimensionalidad en entornos con espacios de estados continuos o muy grandes (como simulaciones físicas en Gymnasium [9]). Para ello, aproxima la función de valor-acción $q(s, a)$ mediante una red neuronal profunda $\hat{q}(s, a; \mathbf{w})$, la cual

Algorithm 11 Sarsa semi-gradiente episódico para estimar $\hat{q} \approx q_*$

- 1: **Entrada:** una parametrización de función de valor de acción diferenciable $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$
- Require:** tamaño de paso $\alpha > 0$, $\varepsilon > 0$ pequeño
- 2: Inicializar los pesos de la función de valor $\mathbf{w} \in \mathbb{R}^d$ arbitrariamente (p.e., $\mathbf{w} = \mathbf{0}$)
- 3: **for** cada episodio **do**
- 4: $S, A \leftarrow$ estado y acción inicial del episodio (p.e., ε -greedy)
- 5: **while** S no sea terminal **do**
- 6: Realizar la acción A , observar R y el siguiente estado S'
- 7: **if** S' es terminal **then**
- 8: $\mathbf{w} \leftarrow \mathbf{w} + \alpha[R - \hat{q}(S, A, \mathbf{w})]\nabla \hat{q}(S, A, \mathbf{w})$
- 9: **ir al siguiente episodio**
- 10: **end if**
- 11: Elegir A' como una función de $\hat{q}(S', \cdot, \mathbf{w})$ (p.e., ε -greedy)
- 12: $\mathbf{w} \leftarrow \mathbf{w} + \alpha[R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})]\nabla \hat{q}(S, A, \mathbf{w})$
- 13: $S \leftarrow S'$
- 14: $A \leftarrow A'$
- 15: **end while**
- 16: **end for**

recibe el estado como entrada y devuelve los valores estimados para todas las acciones posibles.

Dado que el uso de redes neuronales en el aprendizaje por refuerzo tradicional a menudo conduce a la inestabilidad, el algoritmo DQN original [3] introduce dos modificaciones en su arquitectura para estabilizar el aprendizaje:

- **Experience Replay (Memoria de repetición):** Almacena las transiciones pasadas $(s_t, a_t, r_t, s_{t+1}, done)$ en un buffer de capacidad finita N . En lugar de aprender de las muestras de forma secuencial, la red se entrena tomando lotes aleatorios (*minibatches*) de esta memoria. Este mecanismo es crucial porque rompe la fuerte correlación temporal que existe en las trayectorias de los episodios, evitando que la red se sobreajuste a secuencias recientes [3].
- **Target Network (Red Objetivo):** En los métodos de diferencia temporal, el valor objetivo y depende de los mismos pesos $\mathbf{w}$ que se están actualizando. DQN soluciona esto introduciendo una segunda red neuronal $\hat{q}(s, a; \mathbf{w}^-)$, idéntica en arquitectura, pero con sus parámetros $\mathbf{w}^-$ congelados. Estos parámetros solo se sincronizan con los de la red principal cada C pasos. Esto proporciona una diana fija temporalmente, sin esta separación, cada actualización de $\mathbf{w}$ desplazaría simultáneamente el objetivo que se persigue, creando un bucle de retroalimentación positiva que hace que los valores Q crezcan sin control hasta divergir [3].

DQN es un algoritmo *off-policy* que permite escalar Q-Learning a problemas complejos de la vida real, como la evasión meteorológica en aviación [4]. No obstante, hereda de Q-Learning un sesgo sistemático de sobreestimación, el cual suele mitigarse en la literatura actual mediante variantes avan-

Algorithm 12 Deep Q-Learning con Experience Replay

```

1: Inicializar memoria de repetición  $D$  con capacidad  $N$ 
2: Inicializar red  $Q$  con pesos aleatorios  $\mathbf{w}$ 
3: Inicializar red objetivo con  $\mathbf{w}^- \leftarrow \mathbf{w}$ 
4: for cada episodio do
5:   Inicializar estado  $s_1$ 
6:   for  $t = 1$  hasta  $T$  do
7:     Con probabilidad  $\epsilon$  elegir acción aleatoria; si no,
        $a_t = \arg \max_a \hat{q}(s_t, a; \mathbf{w})$ 
8:     Ejecutar  $a_t$ , observar  $r_t, s_{t+1}$  y  $done$ 
9:     Guardar  $(s_t, a_t, r_t, s_{t+1}, done)$  en  $D$ 
10:    Muestrear un minibatch aleatorio de  $D$ 
11:    for cada transición  $(s_j, a_j, r_j, s_{j+1}, done_j)$  do
12:      if  $done_j$  then
13:         $y_j \leftarrow r_j$ 
14:      else
15:         $y_j \leftarrow r_j + \gamma \max_{a'} \hat{q}(s_{j+1}, a'; \mathbf{w}^-)$ 
16:      end if
17:    end for
18:    Actualizar  $\mathbf{w}$  con un paso de gradiente sobre  $(y_j - \hat{q}(s_j, a_j; \mathbf{w}))^2$ 
19:    Cada  $C$  pasos:  $\mathbf{w}^- \leftarrow \mathbf{w}$ 
20:  end for
21: end for

```

zadas como **Double DQN** [10].

2.4. Evaluación/Experimentos

2.4.1. Escenarios experimentales

Las pruebas se realizaron con la librería Gymnasium y semilla fija (SEED=2024) para garantizar reproducibilidad, también se tomaron medidas para garantizar la reproducibilidad en cuanto a la inicialización y operaciones de las redes neuronales. Se estudiaron tres entornos representativos:

- ▶ **Taxi-v3 (discreto):** 500 estados, 6 acciones. Recompensas: -1 por paso, $+20$ entrega exitosa, -10 acción ilegal. Entrenamiento de 100 000 episodios para comparar MC, SARSA y Q-Learning.
- ▶ **CliffWalking-v1 (discreto):** grid 4×12 (48 estados), 4 acciones. Recompensas: -1 por paso, -100 caída al acantilado y 0 al llegar. Entrenamiento de 10 000 episodios para comparar Q-Learning vs SARSA.
- ▶ **LunarLander-v3 (continuo):** estado de 8 variables continuas y 4 acciones discretas: 0 (no hacer nada), 1 (motor lateral izquierdo), 2 (motor principal), 3 (motor lateral derecho). Las recompensas son negativas al gastar combustible (es decir, desplazarse por el escenario) y al estrellar la nave. Son positivas al aterrizar correctamente (alrededor de $+100$ puntos) y dentro del espacio designado (unos $+40$ puntos). Elegimos el espacio de acciones discreto porque DQN y SARSA-SG están formulados para acciones discretas, usar el espacio continuo exigiría métodos de política/actor-crítico (p.ej., DDPG/PPO). Entrenamiento de 5 000 episodios para DQN y SARSA semi-gradiente.

El cambio de escenario a LunarLander-v3 pone de manifiesto

la necesidad de usar métodos aproximados. Esto es debido a que el espacio de estados es continuo donde la posición, velocidad y el ángulo de la nave son variables continuas. Esto **no** es posible de modelar mediante una tabla Q por lo que es imperativo aproximar un estado mediante alguna técnica. Entre ellas destacan el tiling o la utilizada en este trabajo, la red neuronal. Se decidió usar esta última por su mayor complejidad la cual le permite tratar con entornos más complejos.

2.4.2. Configuración Hiperparámetros

Con el objetivo de comprobar distintos comportamientos en el mismo entorno bajo la misma semilla, se utilizaron distintas configuraciones en los experimentos que conrresponden a distintos comportamientos.

Parámetros en Entornos Tabulares (Taxi-v3 y CliffWalking-v1)

En los entornos de representación tabular, el objetivo es estudiar la convergencia hacia la política óptima en espacios de estados discretos.

- ▶ La tasa de aprendizaje (α) controla qué tan rápido el agente actualiza sus valores Q ante nueva información. Un valor elevado puede acelerar la convergencia inicial pero generar inestabilidad, mientras que un valor bajo asegura una convergencia más suave y robusta. Se usará 0.1 como valor estándar por esta razón.
- ▶ El parámetro (ϵ) determina el grado de exploración, permitiendo que el agente descubra nuevas rutas, mientras que el decaimiento (decay) es una estrategia crucial para reducir gradualmente esta exploración y permitir que el agente explote su conocimiento experto conforme avanza el entrenamiento. De esta manera, ϵ baja a medida que aumenta el número de episodios.
- ▶ El factor de descuento (γ) dicta la importancia de las recompensas futuras frente a las inmediatas. Un γ cercano a 1.0 es vital para que, por ejemplo, el taxista priorice completar la entrega final (recompensa de $+20$) a pesar de los costos negativos acumulados en cada paso del trayecto.

Parámetros de Aproximación: DQN y SARSA Semi-gradiente

En entornos de estados continuos (LunarLander-v3), el objetivo es estudiar cómo las redes neuronales generalizan el conocimiento a través de una función de valor aproximada $\hat{q}(s, a, \mathbf{w})$.

▶ Arquitectura de la Red:

- Capacidad de la red (hidden_size): Define la complejidad de las funciones que el agente puede aprender.
- Profundidad de la red (num_hidden_layers): Define el número de capas ocultas entre la entrada y la salida. Una mayor profundidad permite que el agente realice una abstracción jerárquica, las primeras capas procesan datos crudos (posición, velocidad), mientras que las capas más profundas pueden inferir conceptos físicos más complejos (como la inercia o la relación entre el ángulo y la gravedad). Sin embargo, para un problema sencillo, añadir demasiadas capas puede resultar

contraproducente debido al riesgo de overfitting.

► Mecanismos de estabilidad (DQN):

- Tamaño del Replay Buffer: Controla cuántas transiciones pasadas se almacenan para samplear minibatches aleatorios. Un buffer pequeño actualiza con experiencias recientes y correlacionadas, mientras que uno grande diversifica temporalmente el dataset de entrenamiento, reduciendo la varianza de las actualizaciones.
- Frecuencia de actualización de la Target Network: Determina cada cuántos pasos se copia $\mathbf{w}^- \leftarrow \mathbf{w}$. Valores pequeños de C reducen el beneficio de estabilidad (target casi siempre igual a la red principal), mientras que valores grandes hacen que el target quede obsoleto.

Parámetros base utilizados en los experimentos:

- **Tabulares (Taxi):** ϵ -greedy con decaimiento $\epsilon_t = \min(1, 1000/(t+1))$, $\alpha = 0,1$, $\gamma = 1,0$.
- **Tabulares (Cliff):** ϵ -greedy con el mismo decaimiento, $\alpha = 0,1$, $\gamma = 0,99$.
- **DQN (base):** $\alpha = 0,0005$, $\gamma = 0,99$, buffer 50k, batch 64, target update $C = 2000$, red 64×64 , ϵ decay 0,999 (mínimo 0.01).
- **SARSA-SG (base):** $\alpha = 0,001$, $\gamma = 0,99$, red 64×64 , ϵ decay 0,995 (mínimo 0.01).

El objetivo será modificar estos hiperparámetros de manera acorde a lo comentado en este apartado para intentar obtener el mejor rendimiento posible consecuencia de observar distintos comportamientos.

2.4.3. Métricas

Para evaluar el aprendizaje se midieron tres indicadores:

- **Recompensa media acumulada por episodio:** $\bar{R}_t = \frac{1}{t} \sum_{i=1}^t R_i$. la gráfica resultante de este indicador es clave para entender si el agente está aprendiendo (adquiere más recompensas hasta llegar a la secuencia óptima). En entornos con fuertes penalizaciones iniciales (CliffWalking) esta métrica puede permanecer muy negativa.
- **Longitud media de episodios:** número de pasos por episodio (se reporta la media de los últimos 1000 episodios). Este es otro indicador informativo debido a que la longitud del episodio decrece conforme el agente aprende a resolver la tarea más eficientemente. Además, usamos una ventana deslizante en lugar de media acumulada porque la longitud puede oscilar fuertemente al inicio y esta ventana refleja mejor el comportamiento reciente del agente.
- **Pérdida (solo DQN):** evolución de la loss Huber para analizar estabilidad del entrenamiento.

2.4.4. Resultados y Análisis Crítico

Taxi-v3 (control tabular)

Como se aprecia en la Figura 2.5, la comparación conjunta (misma configuración de hiperparámetros) confirma que los métodos TD superan claramente a Monte Carlo en velocidad de convergencia.

Algoritmo	Recompensa media final	Longitud media final (pasos)
MC On-Policy	-138.82	113.73
MC Off-Policy	-22.32	13.69
SARSA	-5.34	13.04
Q-Learning	-4.47	13.06

Tabla 2.1: Resultados en Taxi-v3 (100k episodios).

Los métodos TD convergen en pocos miles de episodios, mientras que MC on-policy queda anclado en episodios largos, como se ve en la Figura 2.6. Q-Learning y SARSA terminan prácticamente empatados en Taxi-v3 (recompensa -4.47 vs -5.34 y longitud 13.06 vs 13.04), lo que indica que en este entorno la diferencia on/off-policy apenas se nota, la dinámica no penaliza de forma fuerte la exploración. Por eso, para analizar con más claridad esa diferencia, se evalúa después CliffWalking, donde el riesgo de caer introduce un contraste real entre ambos métodos.

La razón por la que los métodos de Monte Carlo no convergen a la velocidad de los métodos de DT es porque mientras que MC requiere episodios completos para actualizar la tabla Q , SARSA y Q-Learning actualiza en cada paso, lo que permite aprender a evitar penalizaciones severas (-10 por acciones ilegales) desde los primeros episodios. La longitud media final de pasos es de 13 para todos los métodos excepto MC On-Policy, lo que indica que el agente ha aprendido a realizar la secuencia correcta de acciones para completar la tarea eficientemente, ya que estamos en un mundo con 25 casillas (grid de 5×5).

Por otro lado, Monte Carlo solo actualiza al final del episodio, por lo que en episodios largos, que se dan al principio del aprendizaje, el agente no recibe retroalimentación hasta que se completa la tarea o se alcanza el límite de pasos.

En Q-Learning, un ϵ fijo muy bajo (0.01) da recompensas positivas (6.31), mientras que $\gamma = 0,05$ no aprende (longitud media 82.97 pasos) porque el descuento es tan fuerte que la recompensa final no se propaga hacia atrás, tal como muestra la curva de la Figura 2.1 sobre la recompensa media por episodio. Esto mismo se puede apreciar en la gráfica sobre la longitud media de episodios (Figura 2.2, donde con un ϵ fijo se converge a la optimalidad antes mientras que con $\gamma = 0,05$ nunca converge. La razón por la que un epsilon fijo funciona bien en este entorno es que al ser tan simple, el agente no requiere de una exploración extensa.

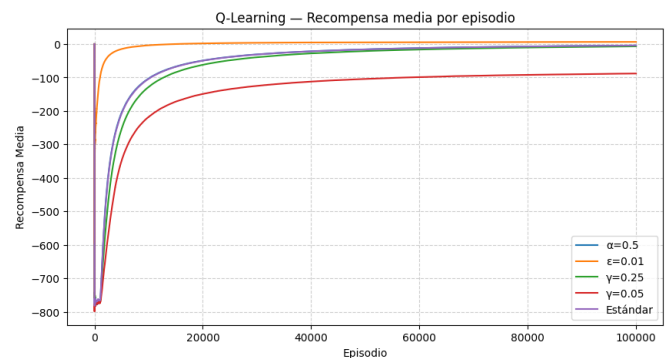


Figura 2.1: Taxi-v3: Recompensa media por episodio. Q-Learning con variaciones de α , ϵ y γ .

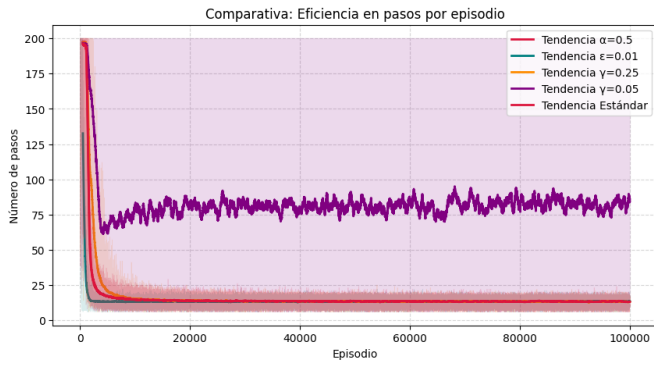


Figura 2.2: Taxi-v3: Longitud media por episodio. Q-Learning con variaciones de α , ϵ y γ .

En SARSA ocurre algo similar: $\epsilon = 0,01$ mejora el rendimiento (recompensa 6.26, 13.25 pasos) y $\gamma = 0,25$ lo degrada claramente (recompensa -159.61, 112.51 pasos) al reducir demasiado la importancia del retorno futuro, visible en la Figura 2.3 y la Figura 2.4.

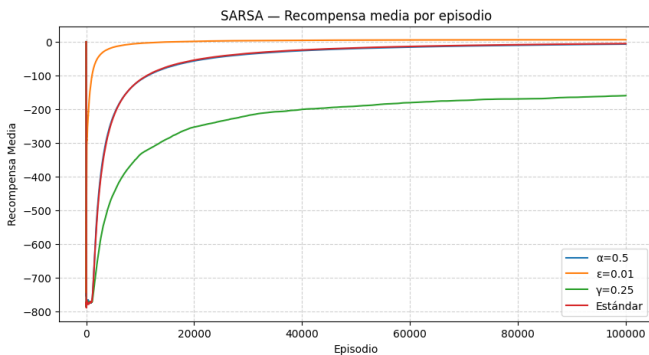


Figura 2.3: Taxi-v3: Recompensa media por episodio. SARSA con variaciones de α , ϵ y γ .

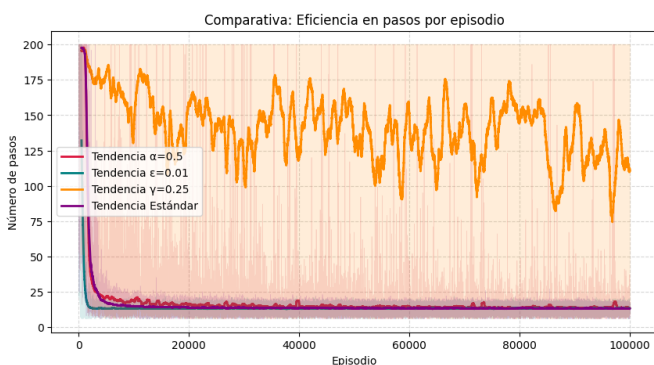


Figura 2.4: Taxi-v3: Longitud media por episodio. SARSA con variaciones de α , ϵ y γ .

Por otro lado, la diferencia entre MC on-policy y off-policy es muy clara, con on-policy el agente mantiene mucha exploración y los episodios permanecen largos, mientras que off-policy puede seguir una política objetivo más greedy y reduce rápidamente la longitud. Esto se aprecia en la Figura 2.6, donde MC on-policy se estabiliza alrededor de 110–120 pasos, y MC off-policy cae a valores cercanos a 14–15 pasos, alineados con SARSA y Q-Learning.

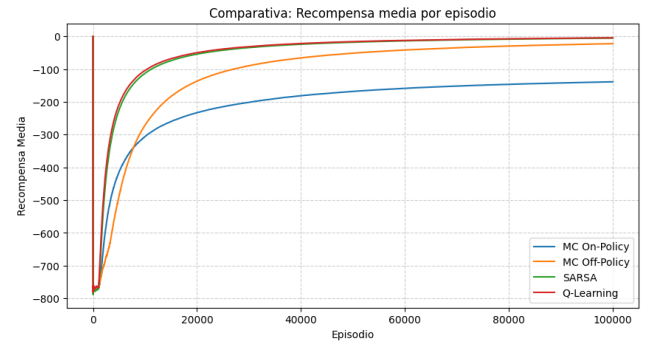


Figura 2.5: Taxi-v3: recompensa media por episodio (MC on/off, SARSA y Q-Learning).

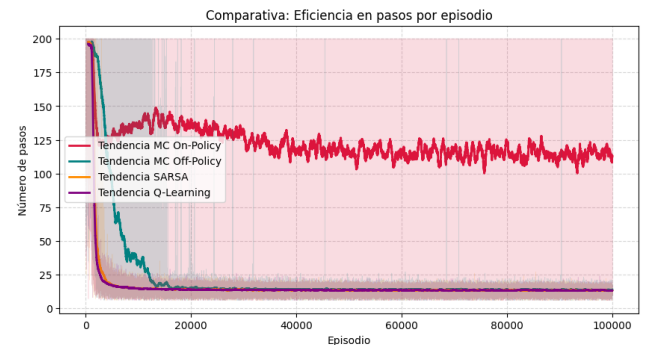


Figura 2.6: Taxi-v3: eficiencia en pasos por episodio (tendencias suavizadas).

CliffWalking (riesgo vs optimalidad)

Aquí se observa la diferencia on-policy/off-policy. Q-Learning converge a la ruta óptima (13 pasos) en el episodio greedy, mientras SARSA aprende la ruta segura (17 pasos) debido a su naturaleza on-policy. La ruta de 17 pasos de SARSA se aleja una celda del borde del acantilado. Este comportamiento es subóptimo en términos de distancia, pero óptimo en términos de recompensa esperada cuando existe exploración ($\epsilon > 0$), ya que minimiza la probabilidad de caer al acantilado por una acción aleatoria.

La media acumulada refleja muchas caídas iniciales porque al explorar el agente cae con frecuencia al acantilado y recibe una penalización fuerte (-100), lo que mantiene la media muy negativa durante los primeros miles de episodios (Figura 2.7). Aparentemente resulta paradójico que SARSA, que aprende la ruta más segura, acumule una media peor que Q-Learning (-8025 vs -6845). La explicación es que Q-Learning, al ser off-policy, propaga el valor óptimo más rápido y acorta su fase de exploración con caídas, SARSA, al evaluarse con la acción que realmente tomará, tarda más en estabilizar su política y acumula más penalizaciones durante esa fase inicial más larga.

- ▶ Q-Learning: recompensa media final -6845.23, longitud media 17.08 pasos.
- ▶ SARSA: recompensa media final -8025.64, longitud media 19.33 pasos.

Con $\epsilon = 0,01$ ambos mejoran notablemente: Q-Learning se acerca a 13.36 pasos y SARSA a 15.19 pasos. En Q-Learning,

$\gamma = 0,05$ falla (longitud media > 180 pasos). Ese fallo se debe a que un factor de descuento tan bajo cambia el comportamiento del agente de modo que valora mucho más las recompensas cercanas. Al valorar tan poco las recompensas futuras, el agente no es capaz de propagar el valor positivo del estado final a los estados iniciales.

La eficiencia en pasos durante el entrenamiento se compara en la Figura 2.8. En ella, se aprecia que Q-Learning converge antes que SARSA debido a lo explicado anteriormente sobre la característica on-policy de SARSA.

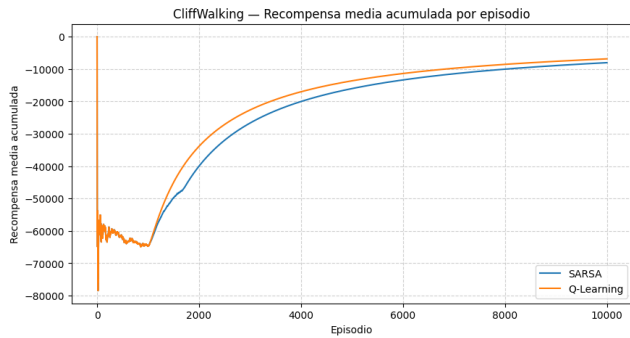


Figura 2.7: CliffWalking-v1: recompensa media acumulada por episodio (SARSA vs Q-Learning).



Figura 2.8: CliffWalking-v1: eficiencia en pasos por episodio (SARSA vs Q-Learning).

LunarLander-v3 (control aproximado)

En el entorno continuo se requiere aproximación de función. Con los hiperparámetros base, SARSA-SG alcanza recompensa superior pero DQN obtiene episodios más cortos:

- SARSA-SG (base 64x64): recompensa media final 176.38, longitud media 295.29 pasos.
- DQN (base): recompensa media final 118.11, longitud media 206.23 pasos.

La Figura 2.11 ilustra la importancia de la Target Network para estabilizar el aprendizaje en métodos de aproximación profunda. Mientras que las configuraciones de $C = 2000$ y $C = 10000$ permiten un crecimiento sostenido de la recompensa hasta lograr aterrizajes exitosos (valores superiores a 100), la configuración con una actualización muy frecuente ($C = 50$) no converge y cada vez alcanza recompensas menores. Esto sucede porque, al cambiar los pesos de la red objetivo casi en cada paso, las metas de aprendizaje dejan de ser estacionarias, lo que provoca que el agente “persiga” un objetivo

que se mueve constantemente, perdiendo toda la estabilidad ganada tras los primeros 500 episodios.

El análisis del tamaño de la memoria de experiencias expuesta en la Figura 2.9 indica que un buffer de 50,000 transiciones permite al agente extraer lotes de entrenamiento con experiencias muy variadas y alejadas en el tiempo, lo que rompe de forma efectiva la correlación temporal de las muestras. Por el contrario, los buffers reducidos (1,000 y 5,000) limitan el aprendizaje a las situaciones más recientes, resultando en una curva de aprendizaje mucho más lenta y con recompensas mediocres que apenas alcanzan el terreno positivo.

La longitud media por episodio de la Figura 2.10 revela que en todas las configuraciones excepto en $C = 50$, hay un patrón de aprendizaje, donde en los primeros episodios el número de pasos por episodio es más alto que en los posteriores debido a que el agente prefiere gastar combustible a estrellarse debido a que se obtiene una recompensa mayor (menos puntos negativos) de esta manera.

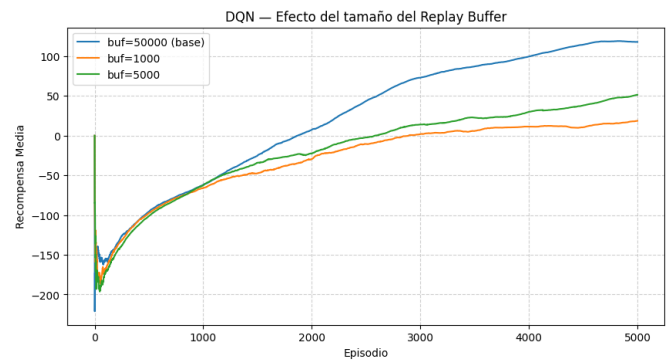


Figura 2.9: LunarLander-v3: Recompensa media por episodio. DQN — efecto del tamaño del Replay Buffer.

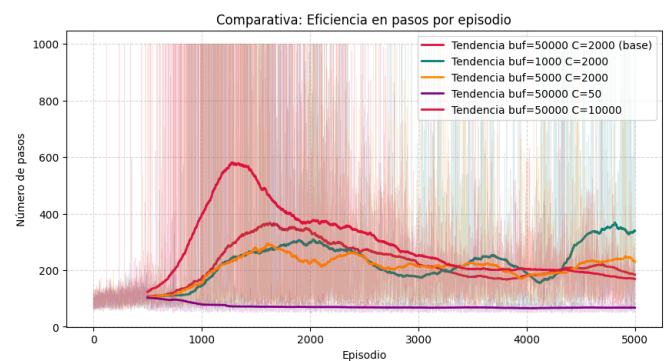


Figura 2.10: LunarLander-v3: Número de pasos por episodio. DQN — efecto del tamaño del Replay Buffer.

En SARSA-SG, la gráfica de recompensa media (Figura 2.12) demuestra que el aprendizaje está directamente ligado al tamaño de las capas ocultas. Las arquitecturas con mayor número de neuronas, específicamente 128x128 y 64x64, presentan las mejores recompensas a medida que suceden los episodios, alcanzando las recompensas finales más altas (cercanas a 175). Por el contrario, la red más pequeña (16x16) es notablemente más lenta y en la Figura 2.13 se puede apreciar como la falta de neuronas para procesar las ocho dimensiones de entrada obliga al agente a realizar vuelos erráticos y muy prolongados, alcanzando una media final de 417.75 pa-

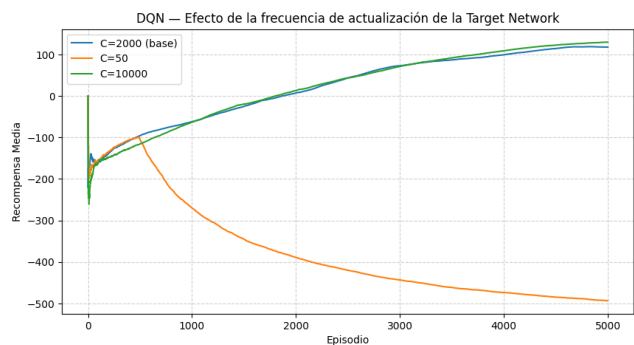


Figura 2.11: LunarLander-v3: DQN — efecto de la frecuencia de actualización de la Target Network (C).

sos, ya que no consigue asimilar la precisión necesaria para un aterrizaje controlado y directo.

Por otro lado, en la Figura 2.12 podemos observar como al introducir una tercera capa oculta se evidencia que la profundidad adicional resulta contraproducente para este problema de control, ya que el rendimiento cae a 118.44 puntos con una red $32 \times 32 \times 32$, frente a los 144,58 que obtiene la red 32×32 , una decremento de 26.14. El aumento de la longitud del episodio a 343.42 pasos respecto a la versión equivalente de dos capas de 319.81 (una diferencia de 23.61 pasos) confirma que la red se vuelve más inestable y difícil de entrenar mediante el algoritmo de semi-gradientes, perdiendo capacidad de generalización frente a arquitecturas más anchas pero menos profundas.

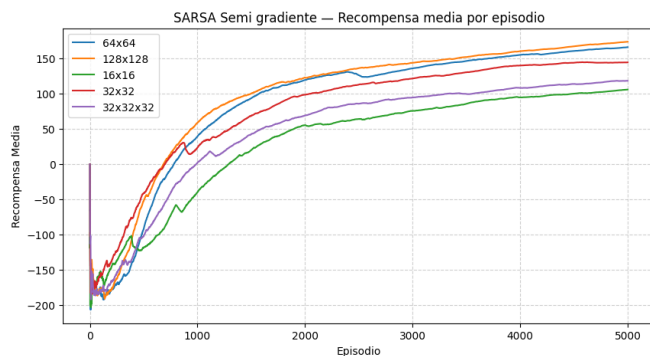


Figura 2.12: LunarLander-v3: Recompensa media por episodio. SARSA-SG - efecto de la arquitectura de red.

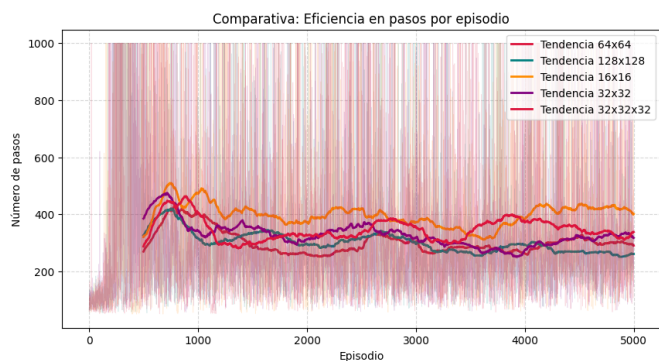


Figura 2.13: LunarLander-v3: Longitud media por episodio. SARSA-SG - efecto de la arquitectura de red.

Para terminar el estudio de este escenario complejo, se com-

paran los dos algoritmos vistos. En la comparativa de recompensas (Figura 2.14, observamos que SARSA Semi-Gradiente no solo aprende más rápido, sino que alcanza un techo de rendimiento superior al de DQN en el horizonte de 5,000 episodios. SARSA-SG termina el experimento con cerca de 175 puntos mientras que DQN con 125 puntos aproximadamente. Sin embargo, ambos agentes no han terminado de converger y DQN parece tener un aprendizaje más lento, pero que puede ser que consiguiera más puntos que SARSA-SG con más episodios. A pesar de esto, se decidió limitar el aprendizaje de agentes a 5000 episodios debido a que cada entrenamiento con SARSA-SG llegaban hasta cerca de la media hora de tiempo de ejecución.

En cuanto al número de pasos promedio mostrado en la Figura 2.15 revela que DQN es un piloto más eficiente. La tendencia de DQN se estabiliza rápidamente cerca de los 200 pasos, lo que indica que el agente ha aprendido a aterrizar siguiendo trayectorias directas y sin maniobras innecesarias. Por el contrario, SARSA SG se estabiliza cerca de los 300 pasos. Esto demuestra que este último es más “cauteloso” o realiza maniobras de corrección más largas en el aire, gana más puntos (posiblemente por aterrizajes más suaves o mayor control de combustible), pero a costa de una mayor duración del episodio.

El aspecto conservador de SARSA-SG se debe a su regla de actualización de los pesos. Estos se ajustan de forma on-policy utilizando la acción siguiente real A' . Esto obliga al agente a aprender el valor de la política que está ejecutando actualmente, lo que suele ser más estable frente al ruido del entorno. Por otro lado, DQN es off-policy y utiliza un operador máx (al igual que en Q-Learning) para estimar el valor futuro, asumiendo siempre que se tomará la mejor acción posible.

En términos absolutos, el entorno LunarLander-v3 se considera resuelto cuando el agente alcanza una recompensa media de ≥ 200 puntos en 100 episodios consecutivos. Ninguno de los dos agentes lo logra en el horizonte de 5000 episodios, SARSA-SG alcanza 176.38 y DQN 118.11. No obstante, ambas curvas siguen una tendencia ascendente al final del entrenamiento, lo que sugiere que con más episodios ambos agentes podrían aproximarse a ese umbral.

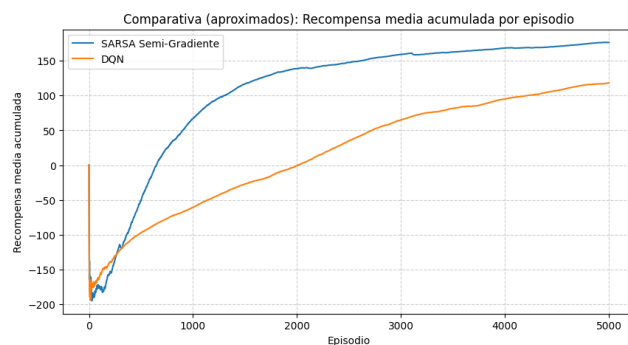


Figura 2.14: LunarLander-v3: recompensa media acumulada por episodio (SARSA-SG vs DQN).

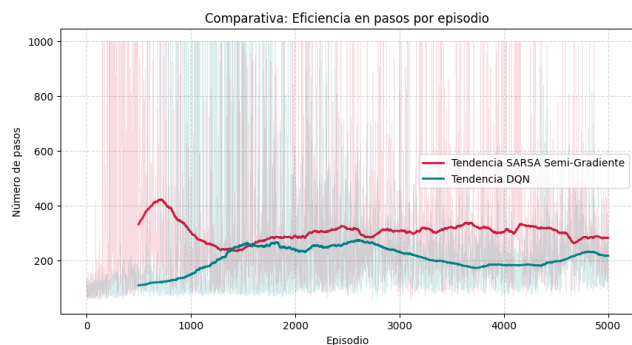


Figura 2.15: LunarLander-v3: eficiencia en pasos por episodio (SARSA-SG vs DQN).

2.5. Conclusiones

Este trabajo ha consistido en análisis de los algoritmos fundamentales del aprendizaje por refuerzo en los que se basan enfoques más complejos utilizados en entornos reales, ya sea en entornos fijos o cambiantes discretos o cambiantes continuos.

En el ámbito del **problema del bandido de k -brazos**, los experimentos demostraron que las estrategias adaptativas superan a las heurísticas simples. **UCB1** se consolidó como el algoritmo más eficiente en términos de arrepentimiento acumulado en todas las distribuciones estudiadas, demostrando que guiar la exploración mediante la incertidumbre es superior al azar puro de ϵ -greedy.

Respecto al **aprendizaje en entornos complejos**, la comparativa en escenarios discretos (Taxi y CliffWalking) subrayó la superioridad de los **métodos de Diferencias Temporales (TD)** sobre Monte Carlo. Mientras que MC requiere episodios completos para aprender, SARSA y Q-Learning logran converger con mayor velocidad al actualizar sus estimaciones paso a paso. Un hallazgo clave fue la distinción entre las políticas *on-policy* y *off-policy*: en entornos de alto riesgo como CliffWalking, **SARSA** adoptó trayectorias más seguras para mitigar el impacto de acciones exploratorias, mientras que **Q-Learning** priorizó la ruta óptima teórica a pesar del peligro de caída.

Finalmente, el paso a entornos continuos como **LunarLander-v3** demostró que la **aproximación de funciones mediante redes neuronales** es imprescindible para el aprendizaje. Se comprobó que mecanismos como el *Experience Replay* y la *Target Network* son pilares fundamentales para la estabilidad de DQN, evitando divergencias durante el entrenamiento. En términos de desempeño, **SARSA-SG** logró acumular mayores recompensas brutas, pero **DQN** demostró ser un piloto más eficiente al resolver la tarea con trayectorias significativamente más cortas.

Reflexión sobre el impacto en el campo

Los algoritmos estudiados en este trabajo constituyen la base sobre la que se construye el aprendizaje por refuerzo moderno. Los métodos tabulares (MC, SARSA, Q-Learning) permiten comprender con precisión los fundamentos teóricos del problema de control, mientras que la transición a métodos apro-

ximados (SARSA semi-gradiente, DQN) ilustra cómo escalar esos fundamentos a problemas del mundo real con espacios de estados intratables. En particular, DQN supuso un punto de inflexión en el campo al demostrar que una red neuronal puede aprender políticas de nivel humano directamente desde observaciones crudas [3], abriendo la puerta a aplicaciones en robótica, conducción autónoma, gestión energética y juegos complejos. Entender sus mecanismos de estabilización (Experience Replay, Target Network) y sus limitaciones (sesgo de sobreestimación) es imprescindible para cualquier profesional que trabaje con RL aplicado.

Limitaciones

El estudio presenta varias limitaciones a tener en cuenta. En primer lugar, el horizonte de entrenamiento en LunarLander-v3 se limitó a 5000 episodios por restricciones de tiempo de cómputo (cada ejecución de SARSA-SG alcanzaba cerca de 30 minutos), lo que impidió que ninguno de los dos agentes alcanzase el umbral de resolución de 200 puntos. En segundo lugar, la búsqueda de hiperparámetros fue manual y acotada, se exploraron configuraciones representativas de cada parámetro de forma individual, sin realizar una búsqueda sistemática (p.ej., grid search o métodos bayesianos) que pudiera identificar combinaciones óptimas. Por último, cada configuración se evaluó bajo una única semilla fija, lo que garantiza reproducibilidad pero no permite estimar la varianza del rendimiento entre distintas inicializaciones.

Líneas de Trabajo Futuro

Como extensiones de este estudio, se identifican las siguientes vías de mejora:

- ▶ Evaluar el rendimiento de algoritmos en entornos **no estacionarios**, donde las distribuciones de recompensa o las dinámicas del mundo cambian con el tiempo. Un tipo de entorno cambiante se ofrece en Gymnasium donde se permite introducir incertidumbre en los movimientos y en el ejemplo del taxi, se puede añadir un parámetro para hacer que el taxi resbale y se desplace dos casillas en vez de una.
- ▶ Explorar métodos de **Gradiente de Política** (como REINFORCE o PPO) para abordar el espacio de acciones continuas en LunarLander, superando las limitaciones de la discretización actual que solo permiten accionar de forma fija los propulsores.
- ▶ Ampliar el estudio a **entornos más complejos** o con un mayor espacio de acciones, como entornos de Atari o MuJoCo, donde el número de acciones posibles y la dimensionalidad del estado exigen una mayor capacidad de generalización de los agentes.

2.5.1. Uso de Herramientas de Inteligencia Artificial

En el desarrollo de este trabajo se han utilizado inteligencia artificial en los siguientes ámbitos:

- ▶ Verificación de la corrección de los algoritmos implementados (pseudocódigo, lógica de actualización y coherencia con la teoría).

- ▶ Revisión de los notebooks para asegurar que todos disponen de los enlaces a Colab, las semillas encargadas de la reproducibilidad, celdas de clonado del repositorio e instalación de dependencias correctas.
- ▶ Apoyo y creación del código de visualización, para ajustar colores, formatos y disposición de las figuras generadas en los experimentos.
- ▶ Utilizado para el completado de sugerencias de código durante el desarrollo (Como sucede con GitHub Copilot en VSCode o Gemini en Google Collab).
- ▶ Utilizado como herramienta de consulta teórica, proporcionándole como contexto las transparencias de la asignatura para resolver dudas sobre los algoritmos y conceptos estudiados (Principalmente NotebookLM, pero también se ha hecho uso de otras IA's con este mismo fin).

La implementación de los agentes, el diseño experimental y el análisis de resultados son trabajo original del grupo.

Bibliografía

- [1] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.
- [2] Tze Leung Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6(1):4–22, 1985.
- [3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [4] Adam Posker. *Evaluating the potential of Deep-Q Networks for weather avoidance in aviation*. PhD thesis, 2025.
- [5] Herbert Robbins. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58(5):527–535, 1952.
- [6] Daniel J. Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, and Zheng Wen. A tutorial on Thompson sampling. *Foundations and Trends in Machine Learning*, 11(1):1–96, 2018.
- [7] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- [8] William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933.
- [9] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- [10] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 2094–2100. AAAI Press, 2016.